

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN CƠ SỞ DỮ LIỆU
ĐỀ TÀI: THIẾT KẾ CƠ SỞ DỮ LIỆU CHO HỆ THỐNG
SÀN THƯƠNG MẠI ĐIỆN TỬ

Giảng viên hướng dẫn: TS. Nguyễn Thị Oanh
Nhóm: 06
Mã lớp: 166216

Sinh viên tham gia:

Nguyễn Thị Thanh Trúc (Nhóm trưởng) – 202417052
Nguyễn Minh Đăng – 202416877
Nguyễn Bá Long – 202416970

Mục lục

PHẦN 1: MÔ TẢ ĐỀ TÀI & NGHIỆP VỤ	3
1.1. Mô tả tổng quan	3
1.2. Mô tả nghiệp vụ.....	3
1.3. Phân tích chức năng hệ thống.....	6
PHẦN 2: THIẾT KẾ CƠ SỞ DỮ LIỆU	9
2.1. Trích xuất danh từ	9
2.2. Phân tích mối quan hệ	9
2.3. Sơ đồ thực thể - liên kết (ERD).....	11
2.4. Chuyển đổi sang Mô hình quan hệ.....	12
2.5. Mô tả chi tiết các bảng dữ liệu	13
PHẦN 3: CÀI ĐẶT CHƯƠNG TRÌNH & TRUY VẤN SQL	23
3.1. Các tính năng đã được cài thực tế.....	23
3.2. Chương trình ứng dụng.....	26
3.3. Danh sách truy vấn SQL và phân tích hiệu năng	33
3.3.1. Phần truy vấn của Sinh viên 1: Nguyễn Minh Đăng – 202416877	33
3.3.2. Phần truy vấn của Sinh viên 2: Nguyễn Bá Long – 202416970	42
3.3.3. Phần truy vấn của Sinh viên 3: Nguyễn Thị Thanh Trúc – 202417052	50
PHẦN 4: ĐÁNH GIÁ & TỔNG KẾT	61
1. Đánh giá kết quả đạt được	61
2. Khó khăn gặp phải & Giải pháp	63
3. Phân công nhiệm vụ chi tiết	64

PHẦN 1: MÔ TẢ ĐỀ TÀI & NGHIỆP VỤ

1.1. Mô tả tổng quan

Trong bối cảnh chuyển đổi số hiện nay, xu hướng mua sắm trực tuyến ngày càng trở nên phổ biến nhờ những ưu điểm nổi bật. Các sàn thương mại điện tử cho phép người dùng mua hàng mọi lúc, mọi nơi, hoạt động liên tục 24/7 mà không bị giới hạn bởi thời gian làm việc truyền thống. Đồng thời, dịch vụ giao hàng tận nơi góp phần giúp người mua tiết kiệm đáng kể thời gian và công sức. Bên cạnh đó, các nền tảng này cung cấp sự đa dạng về sản phẩm và dịch vụ, tạo điều kiện để người bán mở rộng hoạt động kinh doanh một cách nhanh chóng, linh hoạt và không bị giới hạn về mặt địa lý. Hệ thống sản phẩm phong phú trải rộng trên nhiều lĩnh vực, từ hàng tiêu dùng đến các dịch vụ chuyên biệt. Ngoài ra, tính tương tác cao giữa người dùng và hệ thống cho phép khách hàng dễ dàng đánh giá, nhận xét và tìm kiếm sản phẩm phù hợp với nhu cầu. Sàn thương mại điện tử cũng tích hợp nhiều tính năng quan trọng khác như bảo mật thông tin và thanh toán, các chương trình ưu đãi cạnh tranh, cùng với công cụ quản lý tài khoản và thống kê hiệu quả.

Tổng thể, sàn thương mại điện tử không chỉ đơn thuần là một kênh giao dịch trực tuyến mà còn là một hệ thống tích hợp đa chức năng, đáp ứng nhu cầu ngày càng đa dạng của cả người mua lẫn người bán. Dựa trên cơ sở đó, nhóm tiến hành phân tích nghiệp vụ, thiết kế cơ sở dữ liệu và phát triển website nhằm quản lý các thông tin liên quan đến người dùng, sản phẩm, giỏ hàng, đơn hàng, thanh toán và vận chuyển.

1.2. Mô tả nghiệp vụ

Nghiệp vụ 1: Tham gia sàn thương mại

Người dùng (USER) đăng ký tài khoản trên sàn bằng các thông tin cá nhân cơ bản. Mỗi USER được gán một vai trò (ROLE) phù hợp như Customer, Seller, Admin.

Nếu người dùng muốn kinh doanh trên sàn, họ tạo cửa hàng (STORE) để đăng bán sản phẩm.

Sau khi cửa hàng được tạo và được hệ thống chấp nhận, người bán có thể thực hiện hoạt động bán hàng trên sàn.

Nhiệm vụ 2: Đăng bán sản phẩm

Người bán sử dụng cửa hàng của mình để đăng sản phẩm lên hệ thống.

Mỗi sản phẩm (PRODUCT):

- Thuộc một danh mục (CATEGORY)
- Thuộc một thương hiệu (BRAND)
- Có thể có nhiều hình ảnh minh họa (IMAGE)
- Chứa đầy đủ thông tin như tên sản phẩm, mô tả, giá bán và số lượng tồn kho...

Người bán có thể:

- Thêm sản phẩm mới
- Cập nhật thông tin sản phẩm
- Tạm ngừng hoặc tiếp tục kinh doanh sản phẩm

Nhiệm vụ 3: Tìm kiếm và lựa chọn sản phẩm

Người mua truy cập hệ thống để:

- Tìm kiếm sản phẩm theo tên
- Xem sản phẩm theo danh mục
- Xem chi tiết sản phẩm
- So sánh giá và lựa chọn sản phẩm phù hợp

Khi muốn mua hàng, người dùng thêm sản phẩm vào giỏ hàng (CART).

Một giỏ hàng có thể chứa nhiều sản phẩm và người dùng có thể:

- Thay đổi số lượng sản phẩm
- Xóa sản phẩm khỏi giỏ hàng
- Tiếp tục thêm sản phẩm mới

Mỗi người dùng sở hữu một giỏ hàng phục vụ cho việc mua sắm.

Nghiệp vụ 4: Đặt hàng và thanh toán

Từ giỏ hàng, người mua tiến hành đặt hàng để tạo đơn hàng (ORDER).

Mỗi đơn hàng:

- Thuộc về một người dùng
- Có thể chứa nhiều sản phẩm

Sau khi đặt hàng, người mua lựa chọn phương thức thanh toán phù hợp như:

- Thanh toán online
- Thanh toán khi nhận hàng (COD)
- ...

Hệ thống ghi nhận thông tin thanh toán thông qua PAYMENT và liên kết với đơn hàng tương ứng.

Nghiệp vụ 5: Xử lý đơn và giao hàng

Sau khi đơn hàng được tạo:

- Người bán tiếp nhận và xử lý đơn
- Hệ thống tạo thông tin giao hàng (DELIVERY) cho đơn hàng
- Mỗi DELIVERY được gán cho một đơn vị vận chuyển (CARRIER) phù hợp để thực hiện giao hàng.

Mỗi đơn giao hàng:

- Liên kết với một đơn hàng
- Được vận chuyển thông qua một đơn vị vận chuyển (CARRIER)

Đơn vị vận chuyển sẽ phân công nhân viên giao hàng (SHIPPER) thực hiện giao hàng cho người mua. Trong quá trình vận chuyển, SHIPPER cập nhật trạng thái giao hàng và xác nhận khi giao hàng thành công.

Nhiệm vụ 6: Theo dõi và đánh giá sản phẩm

Người mua có thể theo dõi:

- Trạng thái đơn hàng
- Thông tin giao hàng
- Quá trình vận chuyển

Sau khi đơn hàng hoàn tất và giao thành công, người mua có thể đánh giá sản phẩm đã mua.

Mỗi đánh giá:

- Được thực hiện bởi người dùng
- Liên quan đến sản phẩm đã mua
- Bao gồm số sao đánh giá (1-5) và nội dung nhận xét nếu có.

1.3. Phân tích chức năng hệ thống

1. USER (Người dùng)

- Đăng ký tài khoản
- Đăng nhập hệ thống
- Quản lý thông tin cá nhân
- Tìm kiếm sản phẩm theo tên
- Xem sản phẩm theo danh mục
- Xem thông tin chi tiết sản phẩm
- Xem hình ảnh sản phẩm
- Thêm sản phẩm vào giỏ hàng
- Cập nhật số lượng sản phẩm trong giỏ hàng
- Xóa sản phẩm khỏi giỏ hàng
- Tạo đơn hàng từ giỏ hàng
- Lựa chọn phương thức thanh toán
- Xem thông tin thanh toán của đơn hàng
- Theo dõi trạng thái đơn hàng và giao hàng
- Xem lịch sử đơn hàng
- Đánh giá sản phẩm đã mua
- Xem đánh giá của các sản phẩm

2. STORE (Cửa hàng)

- Tạo và cập nhật thông tin cửa hàng
- Đăng sản phẩm mới lên hệ thống
- Chỉnh sửa thông tin sản phẩm
- Cập nhật số lượng tồn kho
- Ngừng kinh doanh sản phẩm
- Quản lý danh sách sản phẩm thuộc cửa hàng
- Xem các đơn hàng liên quan đến sản phẩm của cửa hàng
- Xác nhận xử lý đơn hàng
- Theo dõi thông tin giao hàng của đơn hàng

3. PRODUCT (Quản lý sản phẩm)

- Quản lý thông tin sản phẩm
- Gán sản phẩm vào danh mục (Category)
- Gán thương hiệu (Brand) cho sản phẩm
- Quản lý hình ảnh sản phẩm
- Hiện thị trạng thái còn bán hoặc ngừng bán

4. CART (Giỏ hàng)

- Lưu danh sách sản phẩm người dùng muốn mua
- Quản lý số lượng từng sản phẩm trong giỏ
- Hỗ trợ cập nhật hoặc xóa sản phẩm trước khi đặt hàng

5. ORDER & PAYMENT (Đơn hàng và thanh toán)

- Tạo đơn hàng từ giỏ hàng
- Lưu danh sách sản phẩm trong đơn hàng
- Ghi nhận thông tin thanh toán
- Hỗ trợ nhiều phương thức thanh toán
- Quản lý trạng thái đơn hàng

6. DELIVERY - CARRIER - SHIPPER (Giao hàng)

Carrier

- Nhận yêu cầu giao hàng
- Tiếp nhận đơn vận chuyển từ hệ thống
- Phân công shipper giao hàng

Shipper

- Nhận đơn giao hàng từ Carrier
- Cập nhật trạng thái giao hàng
- Xác nhận giao hàng thành công

Delivery

- Theo dõi quá trình vận chuyển
- Lưu trạng thái giao hàng
- Lưu ngày giao dự kiến và ngày giao thực tế

7. CATEGORY - BRAND - IMAGE

Category:

- Quản lý danh mục sản phẩm

Brand:

- Quản lý thương hiệu sản phẩm

Image:

- Lưu và quản lý hình ảnh của sản phẩm

8. ROLE (Vai trò người dùng)

- Quản lý quyền của người dùng trong hệ thống
- Phân loại người dùng theo vai trò:
 - Người mua
 - Người bán
 - Quản trị viên

PHẦN 2: THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1. Trích xuất danh từ

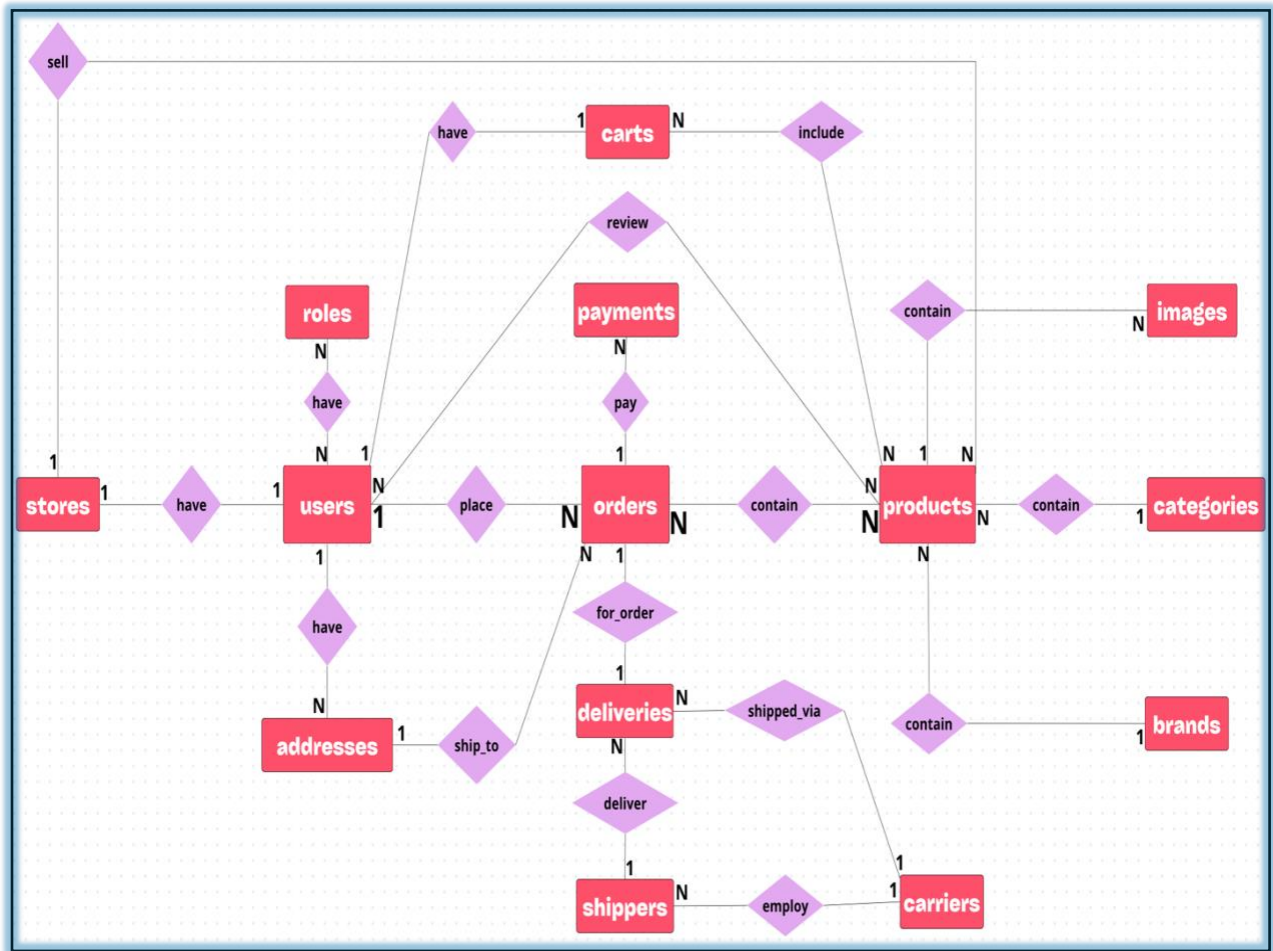
- **Danh từ liên quan đến người/tổ chức:** User (Người dùng), Customer (Khách hàng), Seller (Người bán), Admin (Quản trị viên), Carrier (Đơn vị vận chuyển), Shipper (Nhân viên giao hàng).
- **Danh từ liên quan đến thông tin:** Role (Vai trò), Store (Cửa hàng), Product (Sản phẩm), Category (Danh mục), Brand (Thương hiệu), Image (Hình ảnh sản phẩm), Cart (Giỏ hàng), Order (Đơn hàng), Payment (Thanh toán), Delivery (Giao hàng), Review (Đánh giá sản phẩm).

2.2. Phân tích mối quan hệ

- Một USER có thể có nhiều ROLE và một ROLE có thể được gán cho nhiều USER. Nên quan hệ USER - ROLE là N - N, thông qua quan hệ trung gian USER_ROLE.
- Một USER (nếu là Seller) có thể sở hữu một STORE và mỗi STORE thuộc về một USER. Nên quan hệ USER - STORE là 1 - 1.
- Một STORE có thể đăng bán nhiều PRODUCT và mỗi PRODUCT chỉ thuộc về một STORE. Nên quan hệ STORE - PRODUCT là 1 - N.
- Một CATEGORY chứa nhiều PRODUCT và mỗi PRODUCT chỉ thuộc một CATEGORY. Nên quan hệ CATEGORY - PRODUCT là 1 - N.
- Một BRAND có thể phân phối nhiều PRODUCT và mỗi PRODUCT thuộc một BRAND. Nên quan hệ BRAND - PRODUCT là 1 - N.
- Một PRODUCT có thể có nhiều IMAGE và mỗi IMAGE chỉ thuộc về một PRODUCT. Nên quan hệ PRODUCT - IMAGE là 1 - N.
- Một USER có một CART và mỗi CART thuộc về một USER. Nên quan hệ USER - CART là 1 - 1.

- Một CART có thể chứa nhiều PRODUCT và một PRODUCT có thể nằm trong nhiều CART. Nên quan hệ CART - PRODUCT là N - N, thông qua quan hệ trung gian CART_DETAIL.
- Một USER có thể tạo nhiều ORDER và mỗi ORDER chỉ thuộc về một USER. Nên quan hệ USER - ORDER là 1 - N.
- Một ORDER có thể chứa nhiều PRODUCT và một PRODUCT có thể xuất hiện trong nhiều ORDER. Nên quan hệ ORDER - PRODUCT là N - N, thông qua quan hệ trung gian ORDER_DETAIL.
- Một ORDER có thể được thanh toán bằng nhiều phương thức khác nhau. Mỗi lần thanh toán được lưu thành một PAYMENT riêng biệt và mỗi PAYMENT chỉ thuộc về một ORDER duy nhất. Nên quan hệ ORDER - PAYMENT là 1 - N.
- Một ORDER có một bản ghi DELIVERY và mỗi DELIVERY chỉ thuộc về một ORDER. Nên quan hệ ORDER - DELIVERY là 1 - 1.
- Một SHIPPER có thể thực hiện nhiều DELIVERY và mỗi DELIVERY được thực hiện bởi một SHIPPER. Nên quan hệ SHIPPER - DELIVERY là 1 - N.
- Một CARRIER quản lý nhiều SHIPPER và mỗi SHIPPER thuộc về một CARRIER. Nên quan hệ CARRIER - SHIPPER là 1 - N.
- Một CARRIER có thể tiếp nhận và vận chuyển nhiều DELIVERY, trong khi mỗi DELIVERY chỉ do một CARRIER phụ trách. Nên quan hệ CARRIER - DELIVERY là 1 - N.
- Một USER có thể đánh giá nhiều PRODUCT và một PRODUCT có thể được đánh giá bởi nhiều USER. Nên quan hệ USER - PRODUCT là N - N, thông qua quan hệ trung gian REVIEW.
- Một USER có thể lưu nhiều ADDRESS khác nhau và mỗi ADDRESS chỉ thuộc về một USER duy nhất. Nên quan hệ USER - ADDRESS là 1 - N.
- Một ADDRESS có thể được sử dụng cho nhiều ORDER khác nhau và mỗi ORDER chỉ sử dụng một ADDRESS để giao hàng. Nên quan hệ ADDRESS - ORDER là 1 - N. Trong hệ thống, ADDRESS đóng vai trò là địa chỉ nhận hàng của ORDER.

2.3. Sơ đồ thực thể - liên kết (ERD)



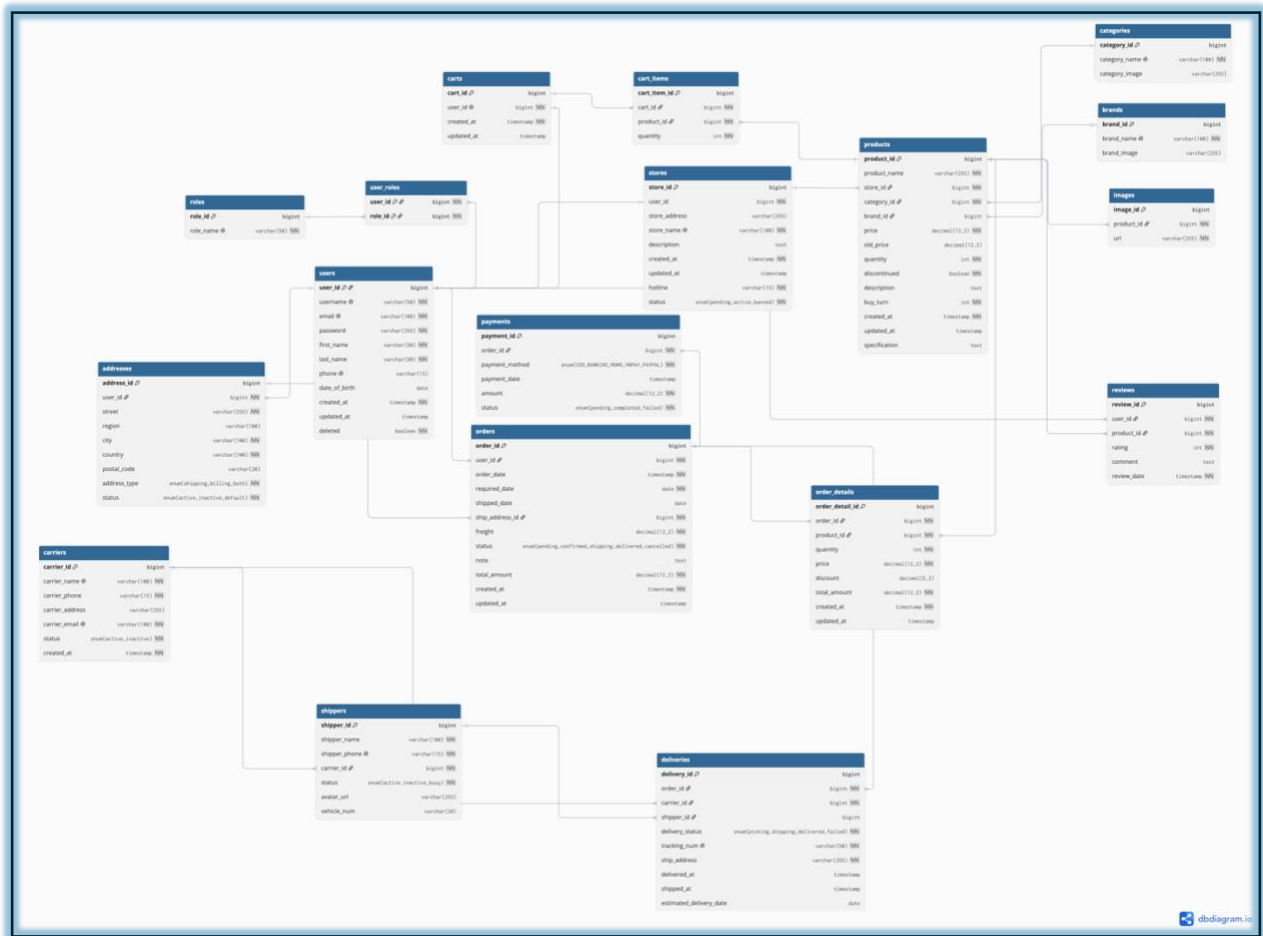
Phần mềm hỗ trợ: Canva.com

Quy ước ký hiệu trong sơ đồ:

- Hình chữ nhật:** Đại diện cho các tập thực thể trong hệ thống.
- Hình thoi:** Minh họa các mối quan hệ (tập liên kết) giữa các thực thể.

Đường dẫn tới sơ đồ ERD: [link_erd](#)

2.4. Chuyển đổi sang Mô hình quan hệ



Phần mềm hỗ trợ: dbdiagram.io

Đường dẫn tới sơ đồ quan hệ: [link_sơ_do_quan_he](#)

2.5. Mô tả chi tiết các bảng dữ liệu

- Bảng **users**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	user_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của người dùng
2	username	VARCHAR(50)	Not null, Unique	Tên đăng nhập của người dùng
3	email	VARCHAR(100)	Not null, Unique	Email tài khoản người dùng
4	password	VARCHAR(255)	Not null	Mật khẩu đã mã hóa của người dùng
5	first_name	VARCHAR(50)	Not null	Tên của người dùng
6	last_name	VARCHAR(50)	Not null	Họ của người dùng
7	phone	VARCHAR(15)	Unique	Số điện thoại người dùng
8	date_of_birth	DATE	Nullable	Ngày sinh của người dùng
9	created_at	TIMESTAMP	Not null	Thời gian tạo tài khoản
10	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật thông tin gần nhất
11	deleted	BOOLEAN	Not null	Trạng thái của tài khoản (true: đã xóa, false: đang hoạt động)

- Bảng **addresses**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	address_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của địa chỉ
2	user_id	BIGINT	Foreign Key (FK), Not null	Mã người dùng sở hữu địa chỉ

3	street	VARCHAR(255)	Not null	Số nhà, tên đường của địa chỉ
4	region	VARCHAR(100)	Nullable	Khu vực/quận/huyện của địa chỉ
5	city	VARCHAR(100)	Not null	Thành phố/tỉnh của địa chỉ
6	country	VARCHAR(100)	Not null	Quốc gia của địa chỉ
7	postal_code	VARCHAR(20)	Nullable	Mã bưu điện của địa chỉ
8	address_type	ENUM('shipping', 'billing', 'both')	Not null	Loại địa chỉ sử dụng để giao hàng, thanh toán hoặc cả hai
9	status	ENUM('active', 'inactive', 'default')	Not null	Trạng thái hoạt động hoặc mặc định của địa chỉ

- Bảng **roles**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	role_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của vai trò
2	role_name	VARCHAR(50)	Not null, Unique	Tên vai trò của người dùng trong hệ thống

- Bảng trung gian **user_roles**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	user_id	BIGINT	Primary Key (PK), Foreign Key (FK)	Mã người dùng được gán vai trò
2	role_id	BIGINT	Primary Key (PK), Foreign Key (FK)	Mã vai trò được gán cho người dùng

- Bảng **stores**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	store_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của cửa hàng
2	user_id	BIGINT	Foreign Key (FK), Not null	Mã người dùng sở hữu cửa hàng
3	store_address	VARCHAR(255)	Nullable	Mã địa chỉ của cửa hàng
4	store_name	VARCHAR(100)	Not null, Unique	Tên cửa hàng
5	description	TEXT	Nullable	Mô tả thông tin cửa hàng
6	created_at	TIMESTAMP	Not null	Thời gian tạo cửa hàng
7	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật thông tin cửa hàng gần nhất
8	hotline	VARCHAR(15)	Not null	Số điện thoại liên hệ của cửa hàng
9	status	ENUM('pending', 'active', 'banned')	Not null	Trạng thái hoạt động của cửa hàng

- Bảng **carts**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	cart_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của giỏ hàng
2	user_id	BIGINT	Foreign Key (FK), Unique, Not null	Mã người dùng sở hữu giỏ hàng
3	created_at	TIMESTAMP	Not null	Thời gian tạo giỏ hàng
4	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật giỏ hàng gần nhất

- Bảng **products**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	product_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của sản phẩm
2	product_name	VARCHAR(255)	Not null	Tên sản phẩm
3	store_id	BIGINT	Foreign Key (FK), Not null	Mã cửa hàng bán sản phẩm
4	category_id	BIGINT	Foreign Key (FK), Not null	Mã danh mục của sản phẩm
5	brand_id	BIGINT	Foreign Key (FK), Nullable	Mã thương hiệu của sản phẩm
6	price	DECIMAL(12,2)	Not null	Giá hiện tại của sản phẩm
7	old_price	DECIMAL(12,2)	Nullable	Giá cũ của sản phẩm trước khi giảm giá
8	quantity	INT	Not null	Số lượng sản phẩm còn trong kho
9	discontinued	BOOLEAN	Not null	Trạng thái ngừng kinh doanh sản phẩm
10	description	TEXT	Nullable	Mô tả chi tiết sản phẩm
11	buy_turn	INT	Not null	Số lượt mua sản phẩm
12	created_at	TIMESTAMP	Not null	Thời gian tạo sản phẩm
13	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật sản phẩm gần nhất
14	specification	TEXT	Nullable	Thông số kỹ thuật hoặc đặc điểm chi tiết của sản phẩm

- Bảng trung gian **cart_items**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	cart_item_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của sản phẩm trong giỏ hàng
2	cart_id	BIGINT	Foreign Key (FK), Not null	Mã giỏ hàng chứa sản phẩm
3	product_id	BIGINT	Foreign Key (FK), Not null	Mã sản phẩm được thêm vào giỏ hàng
4	quantity	INT	Not null	Số lượng sản phẩm trong giỏ hàng

- Bảng **orders**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	order_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của đơn hàng
2	user_id	BIGINT	Foreign Key (FK), Not null	Mã người dùng đặt hàng
3	order_date	TIMESTAMP	Not null	Thời gian đặt hàng
4	required_date	DATE	Not null	Ngày cam kết giao hàng đến khách
5	shipped_date	DATE	Nullable	Ngày thực tế đơn hàng được xuất/giao đi
6	ship_address_id	BIGINT	Foreign Key (FK), Not null	Mã địa chỉ giao hàng của đơn hàng
7	freight	DECIMAL(12,2)	Not null	Phí vận chuyển của đơn hàng
8	status	ENUM('pending', 'confirmed',	Not null	Trạng thái xử lý đơn hàng

		'shipping', 'delivered', 'cancelled')		
9	note	TEXT	Nullable	Ghi chú thêm của khách hàng hoặc hệ thống
10	total_amount	DECIMAL(12,2)	Not null	Tổng tiền thanh toán của đơn hàng
11	created_at	TIMESTAMP	Not null	Thời gian tạo đơn hàng
12	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật trạng thái đơn hàng gần nhất

- Bảng trung gian **order_details**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	order_detail_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của chi tiết đơn hàng
2	order_id	BIGINT	Foreign Key (FK) , Not null	Mã đơn hàng chứa sản phẩm
3	product_id	BIGINT	Foreign Key (FK) , Not null	Mã sản phẩm được đặt mua
4	quantity	INT	Not null	Số lượng sản phẩm được mua
5	price	DECIMAL(12,2)	Not null	Giá sản phẩm tại thời điểm đặt hàng
6	discount	DECIMAL(5,2)	Nullable	Mức giảm giá áp dụng cho sản phẩm
7	total_amount	DECIMAL(12,2)	Not null	Tổng tiền của sản phẩm trong đơn hàng sau giảm giá
8	created_at	TIMESTAMP	Not null	Thời gian tạo chi tiết đơn hàng

9	updated_at	TIMESTAMP	Nullable	Thời gian cập nhật chi tiết đơn hàng gần nhất
---	-------------------	-----------	----------	---

- Bảng **categories**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	category_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của danh mục sản phẩm
2	category_name	VARCHAR(100)	Not null, Unique	Tên danh mục sản phẩm
3	category_image	VARCHAR(255)	Nullable	Đường dẫn hoặc URL hình ảnh của danh mục sản phẩm

- Bảng **brands**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	brand_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của thương hiệu
2	brand_name	VARCHAR(100)	Not null, Unique	Tên thương hiệu sản phẩm
3	brand_image	VARCHAR(255)	Nullable	Đường dẫn hoặc URL hình ảnh của thương hiệu

- Bảng **images**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	image_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của hình ảnh
2	product_id	BIGINT	Foreign Key (FK), Not null	Mã sản phẩm sở hữu hình ảnh

3	url	VARCHAR(255)	Not null	Đường dẫn hoặc URL của hình ảnh sản phẩm
---	------------	--------------	----------	--

- Bảng **deliveries**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	delivery_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của đơn giao hàng
2	order_id	BIGINT	Foreign Key (FK) , Not null	Mã đơn hàng được giao
3	carrier_id	BIGINT	Foreign Key (FK) , Not null	Mã đơn vị vận chuyển phụ trách giao hàng
4	shipper_id	BIGINT	Foreign Key (FK) , Nullable	Mã nhân viên giao hàng thực hiện giao đơn
5	delivery_status	ENUM('picking', 'shipping', 'delivered', 'failed')	Not null	Trạng thái giao hàng
6	tracking_num	VARCHAR(50)	Not null, Unique	Mã vận đơn dùng để theo dõi giao hàng
7	ship_address	VARCHAR(255)	Not null	Địa chỉ giao hàng
8	delivered_at	TIMESTAMP	Nullable	Thời gian giao hàng thành công
9	shipped_at	TIMESTAMP	Nullable	Thời gian đơn hàng được xuất kho/bắt đầu giao
10	estimated_delivery_date	DATE	Nullable	Ngày dự kiến giao hàng

- Bảng **shippers**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	shipper_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của nhân viên giao hàng
2	shipper_name	VARCHAR(100)	Not null	Tên nhân viên giao hàng
3	shipper_phone	VARCHAR(15)	Not null, Unique	Số điện thoại của nhân viên giao hàng
4	carrier_id	BIGINT	Foreign Key (FK), Not null	Mã đơn vị vận chuyển mà shipper thuộc về
5	status	ENUM('active', 'inactive', 'busy')	Not null	Trạng thái hoạt động của shipper
6	avatar_url	VARCHAR(255)	Nullable	Đường dẫn hoặc URL ảnh đại diện của shipper
7	vehicle_num	VARCHAR(20)	Nullable	Biển số phương tiện giao hàng của shipper

- Bảng **carriers**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	carrier_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của đơn vị vận chuyển
2	carrier_name	VARCHAR(100)	Not null, Unique	Tên đơn vị vận chuyển
3	carrier_phone	VARCHAR(15)	Not null	Số điện thoại liên hệ của đơn vị vận chuyển
4	carrier_address	VARCHAR(255)	Nullable	Địa chỉ của đơn vị vận chuyển
5	carrier_email	VARCHAR(100)	Not null, Unique	Email liên hệ của đơn vị vận chuyển

6	status	ENUM('active', 'inactive')	Not null	Trạng thái hoạt động của đơn vị vận chuyển
7	created_at	TIMESTAMP	Not null	Thời gian tạo thông tin đơn vị vận chuyển

- Bảng **payments**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	payment_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của thanh toán
2	order_id	BIGINT	Foreign Key (FK), Not null	Mã đơn hàng được thanh toán
3	payment_method	ENUM('COD', 'BANKING', 'MOMO', 'VNPAY', 'PAYPAL')	Not null	Phương thức thanh toán
4	payment_date	TIMESTAMP	Nullable	Thời gian thực hiện thanh toán
5	amount	DECIMAL(12,2)	Not null	Số tiền thanh toán
6	status	ENUM('pending', 'completed', 'failed')	Not null	Trạng thái thanh toán

- Bảng **reviews**:

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc (Constraint)	Mô tả chi tiết
1	review_id	BIGINT	Primary Key (PK)	Mã định danh duy nhất của đánh giá sản phẩm
2	user_id	BIGINT	Foreign Key (FK), Not null	Mã người dùng thực hiện đánh giá

3	product_id	BIGINT	Foreign Key (FK), Not null	Mã sản phẩm được đánh giá
4	rating	INT	Not null	Số sao đánh giá sản phẩm (1–5)
5	comment	TEXT	Nullable	Nội dung nhận xét của người dùng
6	review_date	TIMESTAMP	Not null	Thời gian người dùng đánh giá sản phẩm

PHẦN 3: CÀI ĐẶT CHƯƠNG TRÌNH & TRUY VẤN SQL

3.1. Các tính năng đã được cài thực tế

Trong giới hạn của môn học Thực hành Cơ sở dữ liệu, nhóm đã tập trung hoàn thiện và cài đặt thành công cấu trúc dữ liệu cốt lõi cùng các phân hệ nghiệp vụ trên môi trường thực tế. Chi tiết các phần đã cài đặt bao gồm:

Nhóm đã triển khai cài đặt thành công cấu trúc vật lý của hệ thống trên hệ quản trị cơ sở dữ liệu PostgreSQL với quy mô **18 bảng dữ liệu đồng bộ**. Toàn bộ các thực thể bao gồm:

- **Phân hệ tài khoản & địa chỉ:** users, addresses, roles, user_roles.
- **Phân hệ gian hàng & sản phẩm:** stores, products, categories, brands, images.
- **Phân hệ giỏ hàng & giao dịch:** carts, cart_items, orders, order_details, payments, deliveries, shippers, carriers.
- **Phân hệ phản hồi:** reviews.

Tất cả các bảng đều được cấu hình chính xác các ràng buộc toàn vẹn dữ liệu ở mức vật lý, bao gồm Khóa chính (Primary Key), Khóa ngoại tham chiếu (Foreign Key), ràng buộc không rỗng (Not Null), ràng buộc duy nhất (Unique) và các ràng buộc miền giá trị kiểu dữ liệu đặc biệt (ENUM, DECIMAL).

Để phục vụ quá trình kiểm thử tính năng và đánh giá hiệu năng câu lệnh SQL một cách khách quan nhất, nhóm đã nạp thành công tập dữ liệu mẫu đồng bộ quy mô vừa thông qua file mã nguồn SQL đính kèm. Tập dữ liệu này mô phỏng các hoạt động mua sắm thực tế bao gồm:

- **300 tài khoản người dùng (users)** hoạt động song song với vai trò người mua (Customer) và người bán (Seller).
- **60 cửa hàng (stores)** đang hoạt động với đầy đủ hotline và địa chỉ riêng biệt.
- **200 sản phẩm (products)** thuộc **15 danh mục (categories)** và **16 thương hiệu (brands)** khác nhau.
- **400 đơn đặt hàng (orders)** phát sinh ngẫu nhiên trong vòng 30 ngày gần nhất với đầy đủ các trạng thái đơn hàng (pending, confirmed, shipping, delivered, cancelled).
- **400 giao dịch thanh toán (payments)** đồng bộ tương ứng với các phương thức (Momo, VNPAY, COD...) cùng thông tin giao hàng chi tiết trong bảng deliveries.

Hệ thống đã cài đặt và cho phép vận hành trơn tru các luồng nghiệp vụ sau trực tiếp bằng các truy vấn SQL:

- **Luồng đăng ký & đăng nhập:** Kiểm tra thông tin định danh, số điện thoại, ngày sinh và phân quyền vai trò cho người dùng.
- **Luồng mua sắm & quản lý giỏ hàng:** Người dùng thêm sản phẩm vào giỏ hàng (cart_items), cập nhật số lượng tồn kho sản phẩm trong bảng products.
- **Luồng xử lý giao dịch & vận chuyển:** Tạo đơn hàng (orders), tự động gán đơn vị vận chuyển (carriers) và shipper phụ trách giao đơn.
- **Luồng tương tác & phản hồi:** Lưu vết các đánh giá sao (rating từ 1-5) và nhận xét của khách hàng đối với các sản phẩm đã mua thành công.

Đặc biệt, để tối ưu hóa xử lý nghiệp vụ vật lý và hỗ trợ giám sát dữ liệu, nhóm đã thiết lập hệ thống các Hàm (Functions), Bộ kích hoạt (Triggers) và Khung nhìn (Views) chuyên biệt:

- **Hàm và Bộ kích hoạt nghiệp vụ (Functions & Triggers):**
 - **Kiểm soát kho tự động:** Thiết lập bộ kích hoạt trigger_update_stock gọi hàm update_product_stock_func() trên bảng order_details. Khi có chi tiết đơn hàng mới được tạo (INSERT), hệ thống tự động kiểm tra

số lượng tồn kho vật lý; nếu đủ điều kiện sẽ tự động trừ kho sản phẩm và cộng dồn lũy kế lượt mua (buy_turn).

- **Tính toán tổng tiền đơn hàng:** Cài đặt hàm `calculate_order_total(target_order_id)` hỗ trợ tự động tính tổng tiền thực tế của đơn hàng bằng cách cộng tổng giá trị các mặt hàng với phí vận chuyển (freight), sau đó cập nhật trực tiếp vào trường `total_amount` của bảng `orders`.
- **Khung nhìn hỗ trợ báo cáo và giám sát (Database Views):**
 - **view_store_performance (Báo cáo hiệu suất cửa hàng):** Tổng hợp dữ liệu động để theo dõi số lượng sản phẩm, số đơn hàng hoàn thành và doanh thu thực tế thu được của từng cửa hàng (chỉ tính các đơn hàng đã giao thành công).
 - **view_delivery_tracking (Theo dõi vận đơn thời gian thực):** Hỗ trợ bộ phận vận hành giám sát trạng thái giao hàng, thông tin đơn vị vận chuyển (carriers), nhân viên giao hàng (shippers) đi kèm số điện thoại liên hệ và các mốc thời gian xuất kho, giao nhận thực tế.

Bên cạnh việc cài đặt cấu trúc bảng và các đối tượng logic trên, nhóm đã triển khai cài đặt thực tế các **Chỉ mục phụ (Secondary Indexes)** trên môi trường PostgreSQL nhằm tối ưu hóa tối đa hiệu năng truy xuất:

- **Chỉ mục trên khóa ngoại (B-Tree Indexes):** Cấu hình trên các trường khóa ngoại thường tham gia phép toán liên kết (như `category_id`, `brand_id`, `store_id`, `shipper_id`) để tăng tốc xử lý cho các truy vấn JOIN.
- **Chỉ mục phức hợp (Composite Indexes):** Thiết lập các chỉ mục kết hợp nhiều trường phục vụ cho bộ lọc và sắp xếp có tần suất cao, bao gồm:
 - `idx_products_status_created` hỗ trợ lọc nhanh trạng thái sản phẩm (discontinued), số lượng tồn kho (quantity) và sắp xếp theo ngày tạo mới nhất (`created_at` DESC).
 - `idx_orders_user_date` hỗ trợ truy vấn lịch sử mua sắm của khách hàng (`user_id`) sắp xếp theo ngày đặt hàng mới nhất (`order_date` DESC).
- **Chỉ mục duy nhất (Unique Index):** Cài đặt chỉ mục `idx_deliveries_tracking_num` trên cột mã vận đơn (`tracking_num`) giúp truy xuất cực nhanh thông tin hành trình giao nhận đơn lẻ.
- **Chỉ mục GIN (Generalized Inverted Index):** Cấu hình chỉ mục `idx_products_name_fts` hỗ trợ tìm kiếm toàn văn (Full-Text Search) không dấu/có dấu trên trường tên sản phẩm (`product_name`).

Hiệu năng thực tế của các chỉ mục này sẽ được nhóm phân tích và đánh giá chi tiết thông qua công cụ EXPLAIN ANALYZE ở các mục tiếp theo.

3.2. Chương trình ứng dụng

Để kiểm thử trực quan tương tác dữ liệu vật lý, nhóm đã xây dựng ứng dụng Desktop bằng Java Swing kết nối trực tiếp với PostgreSQL qua cổng mạng 5432.

Cơ chế kết nối & Bảo mật: Ứng dụng kết nối qua trình điều khiển JDBC tiêu chuẩn. Ngay khi khởi chạy, một hộp thoại bảo mật yêu cầu người dùng nhập tài khoản PostgreSQL với mật khẩu được che dạng **** (JPasswordField). Toàn bộ truy vấn sử dụng PreparedStatement để chống tấn công SQL Injection. Quy trình đặt hàng được quản lý bằng Database Transaction để đảm bảo tính toàn vẹn dữ liệu. Giao diện nút bấm được cấu hình tùy biến để tương thích tốt, không bị khuyết chữ hay lệch viền trên cả macOS và Windows.

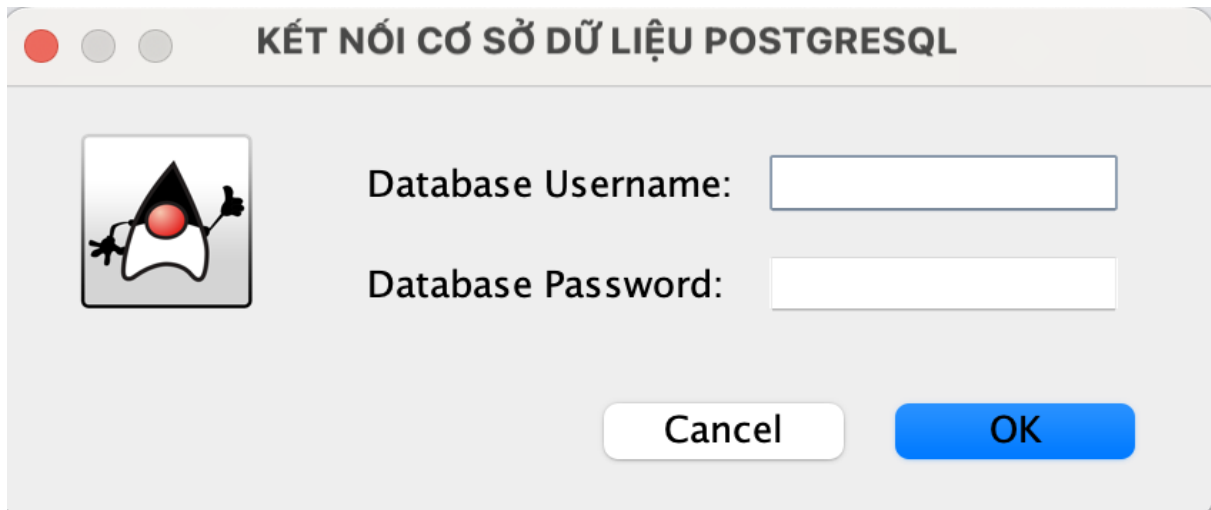
Các phân hệ nghiệp vụ tương tác chính:

- **Xác thực tài khoản & Giỏ hàng:** Hỗ trợ Đăng ký/Đăng nhập tài khoản users, tự động kiểm tra và khởi tạo một giỏ hàng trống tương ứng trong bảng carts nếu chưa tồn tại.
- **Tìm kiếm & Lọc sản phẩm động:** Trang chủ tích hợp bộ lọc nhanh theo danh mục dạng thanh cuộn phẳng và thanh tìm kiếm từ khóa. Lưới sản phẩm hiển thị trực quan các thông số: giá mới, giá gốc, % giảm giá và lượt bán thực tế.
- **Giỏ hàng & Kiểm soát tồn kho:** Cho phép thêm sản phẩm với Spinner không chế số lượng giới hạn tối đa bằng tồn kho thực tế. Giỏ hàng chi tiết hỗ trợ điều chỉnh số lượng, xóa sản phẩm, chọn địa chỉ nhận hàng động từ bảng addresses hoặc thêm nhanh địa chỉ mới qua Modal Dialog.
- **Thanh toán Transaction & Vận đơn tự động:** Khi đặt hàng, hệ thống thực thi một Database Transaction khép kín: tạo đơn hàng (orders), lưu chi tiết (order_details) kích hoạt Trigger tự động trừ kho, tự chọn đối tác vận chuyển (carriers), tự điều phối shipper rảnh rỗi (shippers) và sinh kịch bản giao hàng (deliveries) kèm Biên lai báo cáo thành công.

Mô phỏng chương trình & Kết quả chạy thử:

1. Kịch bản 1: Kết nối Cơ sở dữ liệu vật lý

- *Hình ảnh:* Ảnh chụp hộp thoại nhập Username/Password PostgreSQL khi vừa bật chương trình.

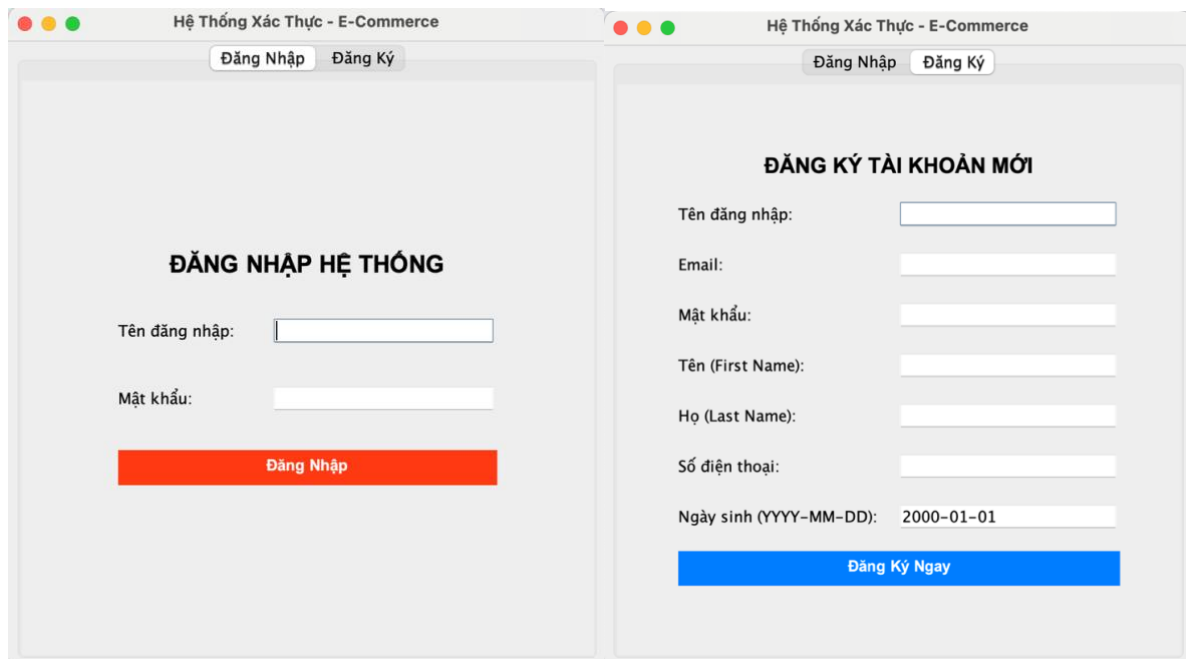


The screenshot shows a window titled "KẾT NỐI CƠ SỞ DỮ LIỆU POSTGRESQL". On the left is a cartoon character of a red ball with a black outline, wearing a white shirt and giving a thumbs up. To the right of the character are two input fields: "Database Username:" and "Database Password:". Below these fields are two buttons: "Cancel" (white with a grey border) and "OK" (blue).

- *Mô tả:* Người dùng câu hình tài khoản kết nối, mật khẩu được che dạng **** để bảo mật.

2. Kịch bản 2: Đăng ký và Đăng nhập người dùng

- *Hình ảnh:* Ảnh chụp màn hình Đăng nhập / Đăng ký.

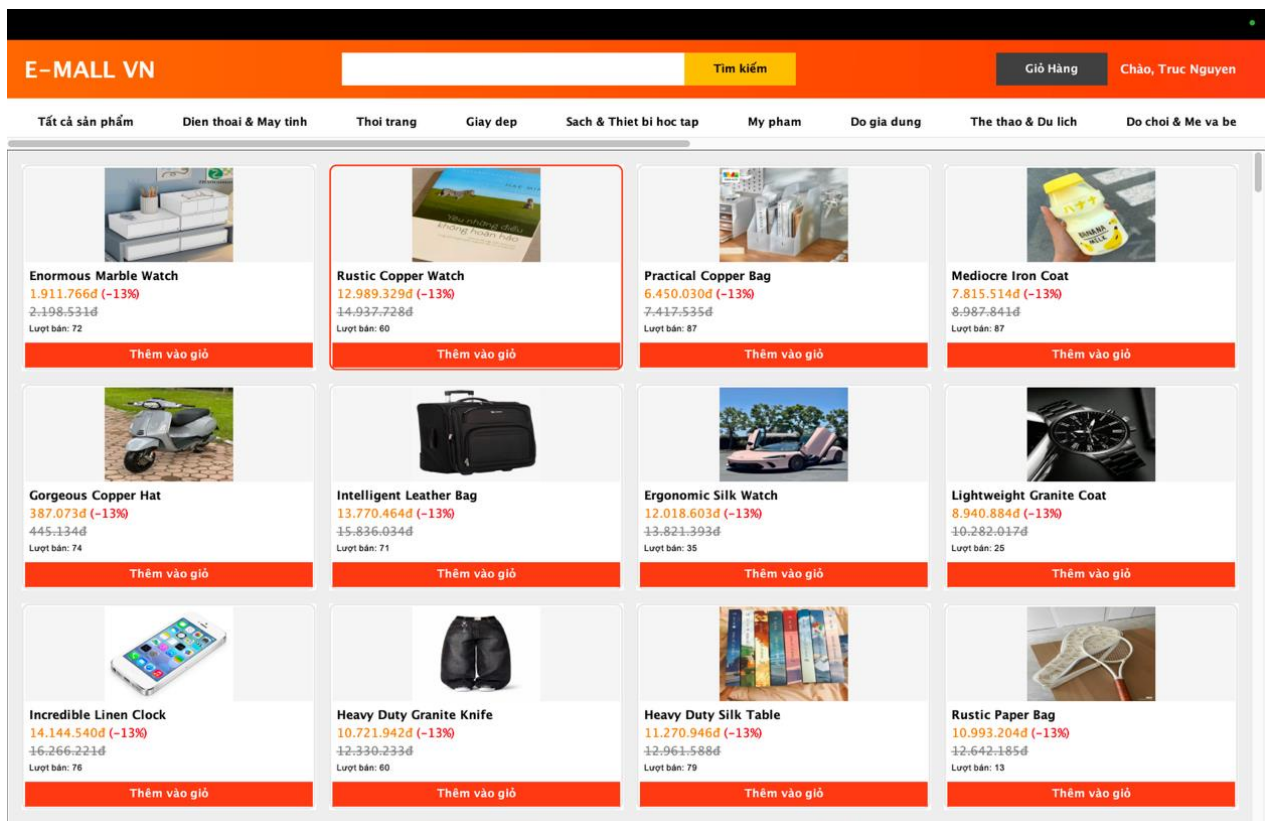


The screenshot displays two side-by-side windows for the "Hệ Thống Xác Thực - E-Commerce". Both windows have tabs for "Đăng Nhập" (Login) and "Đăng Ký" (Register). The left window is titled "ĐĂNG NHẬP HỆ THỐNG" and contains fields for "Tên đăng nhập:" and "Mật khẩu:", with a red "Đăng Nhập" button at the bottom. The right window is titled "ĐĂNG KÝ TÀI KHOẢN MỚI" and contains fields for "Tên đăng nhập:", "Email:", "Mật khẩu:", "Tên (First Name):", "Họ (Last Name):", "Số điện thoại:", and "Ngày sinh (YYYY-MM-DD):" (with the example date "2000-01-01"). A blue "Đăng Ký Ngay" button is at the bottom of the right window.

- *Mô tả:* Đăng nhập bằng tài khoản để truy cập vào hệ thống. Người dùng mới chưa có tài khoản có thể chuyển sang tab Đăng ký để điền thông tin và tạo tài khoản; hệ thống sẽ tự động thêm người dùng mới vào bảng users và khởi tạo một giỏ hàng trống tương ứng trong bảng carts.

3. Kịch bản 3: Giao diện Trang chủ mua sắm

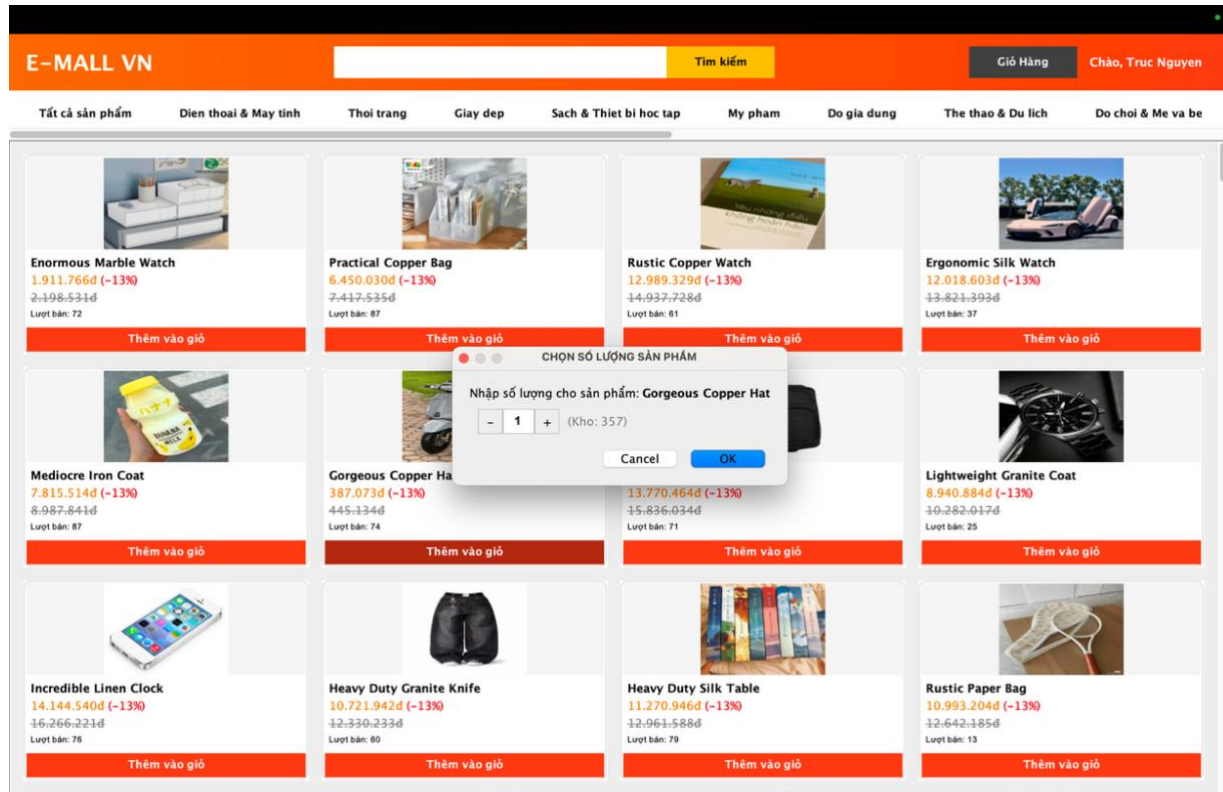
- *Hình ảnh:* Ảnh chụp màn hình giao diện Trang chủ mua sắm E-MALL VN.



- *Mô tả:* Hệ thống hiển thị thanh cuộn ngang danh mục sản phẩm phẳng (Categories) ở phía trên và lưới thẻ sản phẩm trực quan phía dưới. Mỗi thẻ sản phẩm hiển thị đầy đủ thông tin: tên sản phẩm, giá bán khuyến mãi (màu cam), giá gốc (gạch ngang xám), tỷ lệ giảm giá (%) và lượt bán thực tế (buy_turn) được đồng bộ thời gian thực từ database.

4. Kịch bản 4: Tương tác chọn số lượng thêm vào giỏ hàng

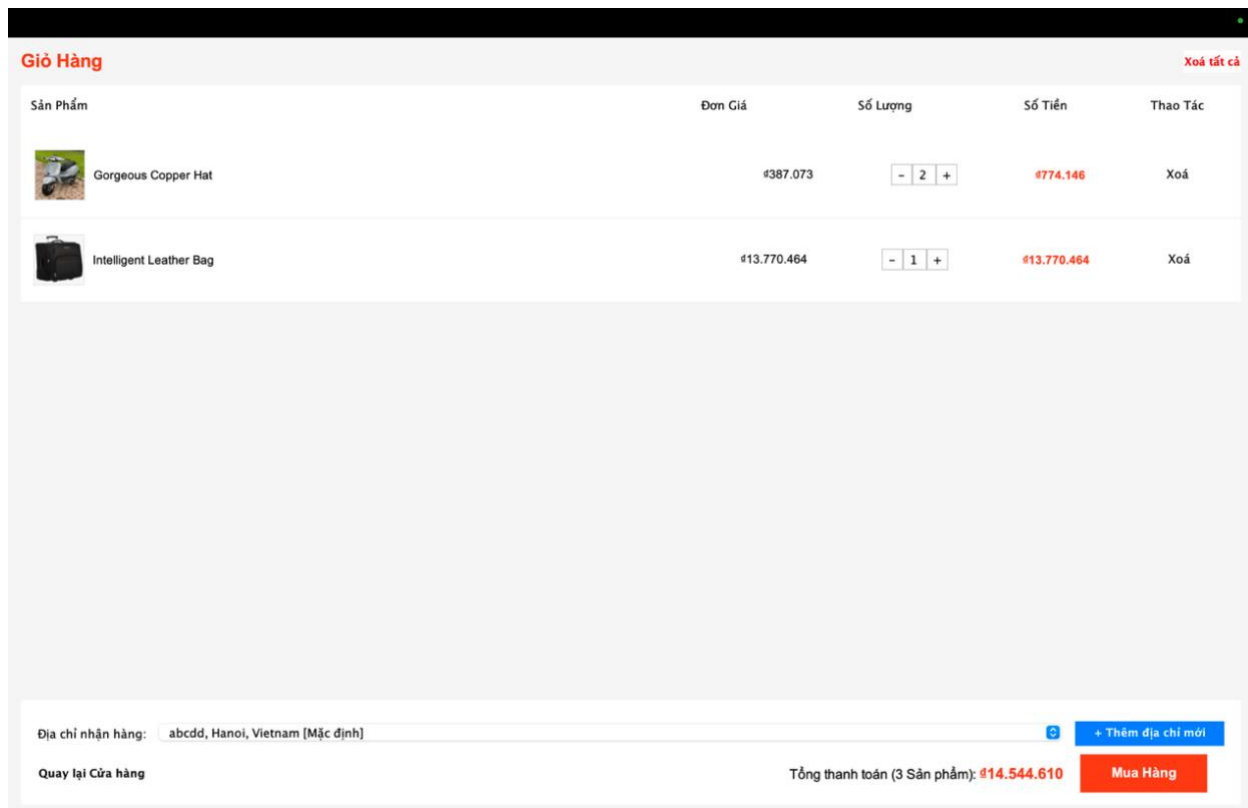
- *Hình ảnh:* Ảnh chụp hộp thoại chọn số lượng sản phẩm khi nhấn nút "Thêm vào giỏ".



- *Mô tả:* Khi người dùng chọn chức năng "Thêm vào giỏ", một hộp thoại phụ sẽ hiển thị. Hệ thống tự động thực hiện truy vấn số lượng tồn kho còn lại của sản phẩm đó trong cơ sở dữ liệu để giới hạn mức số lượng đặt mua thực tế của Spinner, giúp ngăn chặn trường hợp khách hàng đặt mua vượt quá số lượng khả dụng trong kho.

5. Kịch bản 5: Giao diện Giỏ hàng chi tiết và Chọn địa chỉ

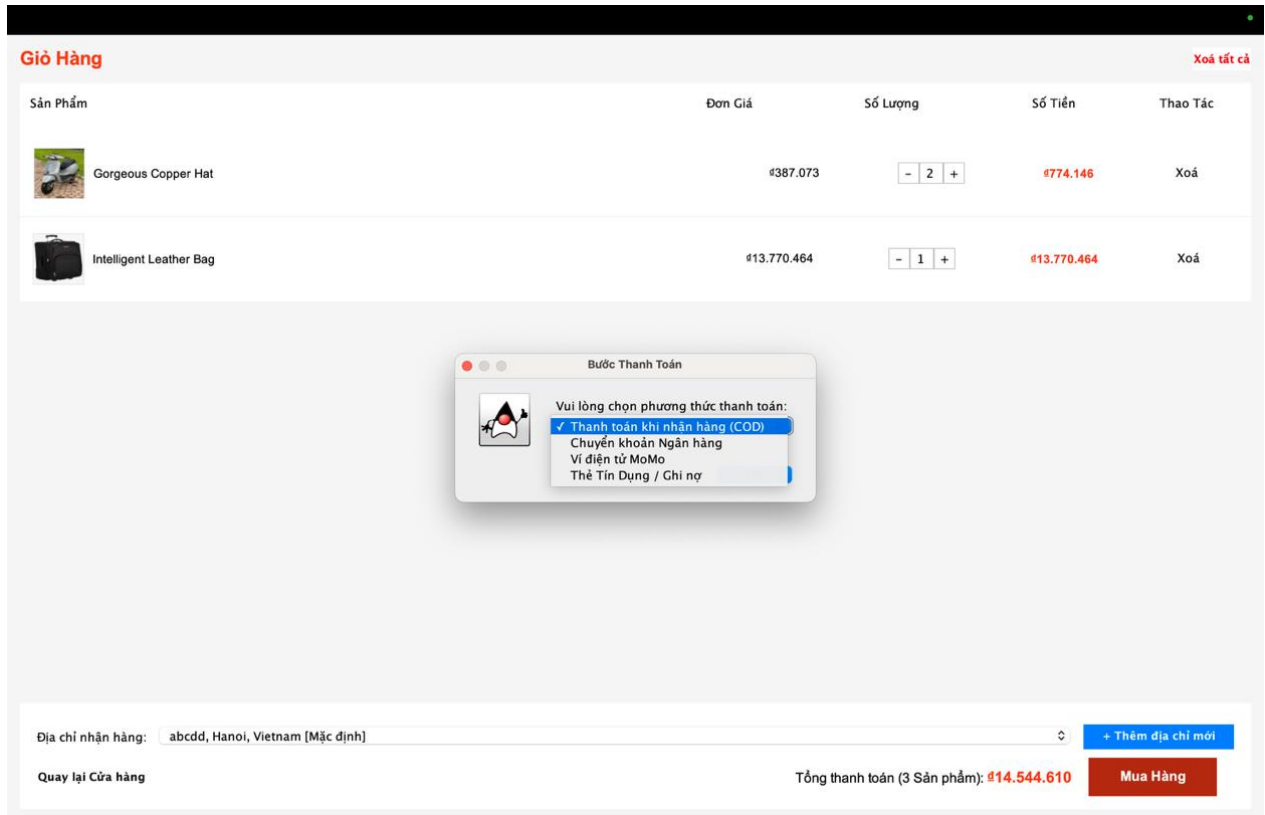
- *Hình ảnh:* Ảnh chụp giao diện màn hình Giỏ hàng chi tiết của người dùng.



- *Mô tả:* Liệt kê danh sách các sản phẩm đã được thêm vào giỏ hàng kèm theo đơn giá, số lượng (tích hợp nút tăng/giảm số lượng nhanh), tính toán thành tiền tương ứng và tùy chọn xóa sản phẩm. Phía dưới giỏ hàng hiển thị hộp chọn Dropdown (JComboBox) chứa danh sách địa chỉ giao nhận được đồng bộ thời gian thực từ bảng addresses của tài khoản đang đăng nhập và nút bấm "+ Thêm địa chỉ mới" hỗ trợ mở hộp thoại thêm nhanh địa chỉ mới trực tiếp vào cơ sở dữ liệu.

6. Kịch bản 6: Xác nhận phương thức thanh toán

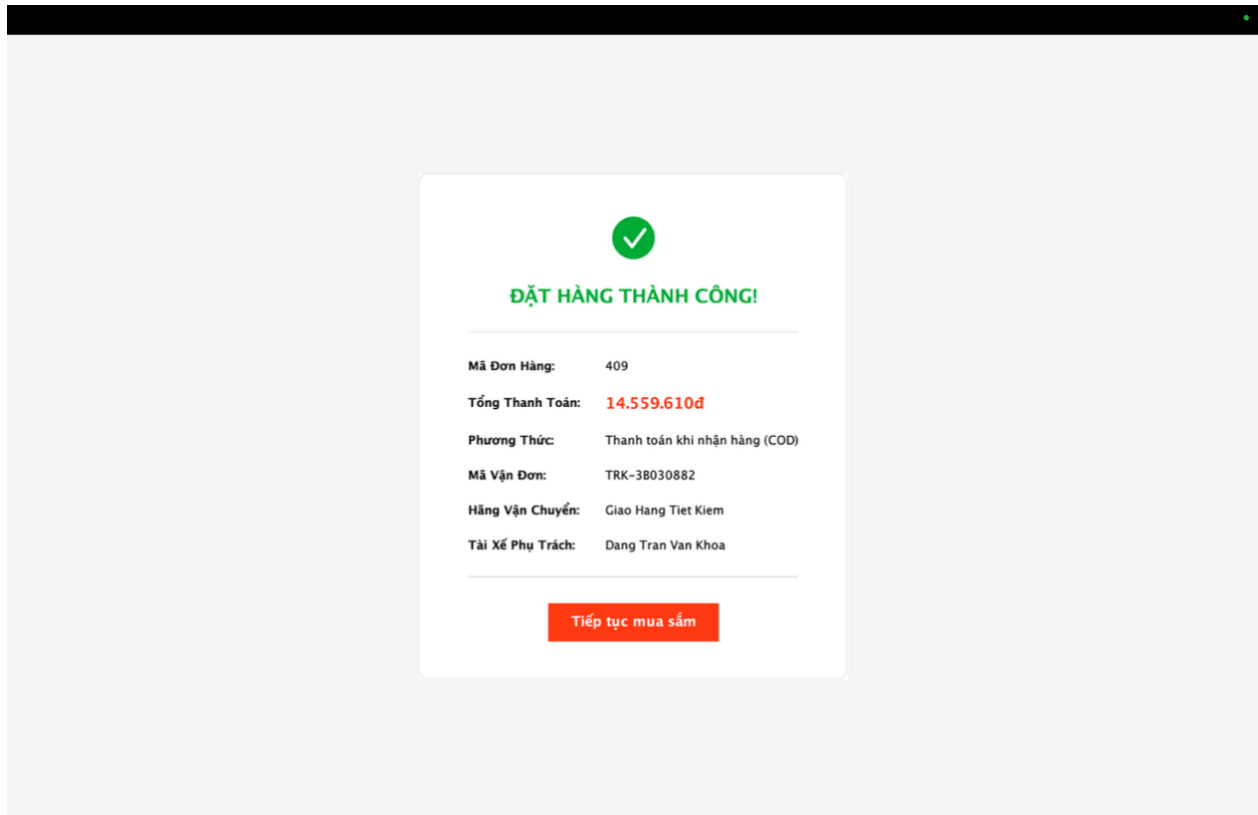
- *Hình ảnh:* Ảnh chụp hộp thoại lựa chọn phương thức thanh toán khi người dùng nhấn nút "Mua Hàng".



- *Mô tả:* Hệ thống hiển thị danh sách các phương thức thanh toán khả dụng được đồng bộ từ kiểu dữ liệu liệt kê (`payment_method_enum`) trong cơ sở dữ liệu để khách hàng lựa chọn phương thức giao dịch mong muốn cho đơn hàng.

7. Kịch bản 7: Thanh toán giao dịch và Điều phối vận đơn tự động

- *Hình ảnh:* Ảnh chụp giao diện Biên lai báo cáo kết quả "Đặt hàng thành công".



- *Mô tả:* Khi người dùng xác nhận đặt hàng, hệ thống thực thi một giao dịch Database Transaction khép kín để bảo toàn tính vẹn toàn dữ liệu: khởi tạo hóa đơn mới trong bảng orders (tính toán tổng tiền đã bao gồm phí vận chuyển), lưu chi tiết đơn hàng sang bảng order_details kích hoạt Trigger tự động trừ kho sản phẩm. Đồng thời, hệ thống tự động gán đối tác vận chuyển đang hoạt động trong bảng carriers, tự động điều phối shipper đang rảnh rỗi trong bảng shippers tham gia nhận đơn và tự động sinh mã vận đơn ngẫu nhiên ghi nhận vào bảng deliveries để phục vụ tiến trình theo dõi giao vận.

3.3. Danh sách truy vấn SQL và phân tích hiệu năng

3.3.1. Phần truy vấn của Sinh viên 1: Nguyễn Minh Đăng – 202416877

1. **Yêu cầu nghiệp vụ** : Khách hàng muốn tìm sản phẩm có tên “Enormous Marble Watch” (không phân biệt tìm kiếm chữ cái hoa thường)

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT *
FROM products
WHERE to_tsvector('english',product_name)
@@ plainto_tsquery('english','Enormous Marble Watch');
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	A2 QUERY PLAN
1	Bitmap Heap Scan on products (cost=17.35..21.61 rows=1 width=226) (actual time=0.041..0.042 rows=1.00 loops=1)
2	Recheck Cond: (to_tsvector('english':regconfig, (product_name)::text) @@ "'enorm" & "marbl" & "watch"::tsquery)
3	Heap Blocks: exact=1
4	Buffers: shared hit=5
5	-> Bitmap Index Scan on idx_products_name_fts (cost=0.00..17.35 rows=1 width=0) (actual time=0.019..0.019 rows=1)
6	Index Cond: (to_tsvector('english':regconfig, (product_name)::text) @@ "'enorm" & "marbl" & "watch"::tsquery)
7	Index Searches: 1
8	Buffers: shared hit=4
9	Planning:
10	Buffers: shared hit=1
11	Planning Time: 0.271 ms
12	Execution Time: 0.070 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Bitmap Index Scan** trên index GIN idx_products_name_fts, sau đó thực hiện **Bitmap Heap Scan** để lấy dữ liệu thực tế. PostgreSQL tự động chuẩn hóa từ khóa thành dạng gốc 'enorm' & 'marbl' & 'watch' thông qua cơ chế stemming của Full Text Search, chỉ cần đọc **5 buffer blocks** để tìm ra kết quả.
 - Thời gian thực thi: Tổng actual time là 0.121 ms — rất nhanh, index GIN được tận dụng hiệu quả, không xảy ra quét tuần tự toàn bảng.

- Đánh giá: Câu truy vấn hoạt động tốt nhờ tận dụng index GIN sẵn có. Planning Time: 66.507 ms cao bất thường do đây là lần đầu index được nạp vào bộ nhớ (dirty=1) — ở các lần truy vấn tiếp theo chỉ số này sẽ giảm đáng kể. Hệ thống hoạt động ổn định và có khả năng mở rộng tốt khi dữ liệu tăng lên.

2. Yêu cầu nghiệp vụ : Khách hàng muốn tìm các sản phẩm khuyến mại có giá gốc cao hơn giá hiện tại

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM products
WHERE old_price > price;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN	
1	Seq Scan on products (cost=0.00..10.50 rows=67 width=226) (actual time=1.576..1.635 rows=200.00 loops=1)
2	Filter: (old_price > price)
3	Buffers: shared hit=8
4	Planning Time: 0.072 ms
5	Execution Time: 1.652 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Seq Scan** (quét tuần tự toàn bảng) vì điều kiện `old_price > price` so sánh trực tiếp giữa 2 cột với nhau trong cùng một bảng — đây là dạng điều kiện mà index thông thường không thể hỗ trợ. Toàn bộ 200 dòng được đọc và không có dòng nào bị loại bởi Filter, tức tất cả sản phẩm đều có khuyến mãi.
 - Thời gian thực thi: Planning Time: 0.084 ms — rất nhanh ở quy mô hiện tại, Planning Time: 0.100 ms cũng rất thấp cho thấy kế hoạch truy vấn đơn giản, không tốn chi phí lập kế hoạch.
 - Đánh giá: Seq Scan trong trường hợp này là hợp lý và tối ưu vì điều kiện so sánh 2 cột không thể dùng index thông thường. Nếu hệ thống cần truy vấn này thường xuyên trên tập dữ liệu lớn, có thể cân nhắc tạo **partial index** hoặc thêm cột `has_discount` BOOLEAN được cập nhật sẵn để tăng tốc lọc.

3. Yêu cầu nghiệp vụ : Khách hàng muốn tìm top 20 sản phẩm “hot” nhất (có lượt bán nhiều nhất)

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM products
ORDER BY buy_turn DESC LIMIT 20;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Limit (cost=15.32..15.37 rows=20 width=226) (actual time=0.096..0.099 rows=20.00 loops=1)
2	Buffers: shared hit=8
3	-> Sort (cost=15.32..15.82 rows=200 width=226) (actual time=0.095..0.097 rows=20.00 loops=1)
4	Sort Key: buy_turn DESC
5	Sort Method: top-N heapsort Memory: 37kB
6	Buffers: shared hit=8
7	-> Seq Scan on products (cost=0.00..10.00 rows=200 width=226) (actual time=0.012..0.030 rows=200.00 loops=1)
8	Buffers: shared hit=8
9	Planning:
10	Buffers: shared hit=4
11	Planning Time: 0.201 ms
12	Execution Time: 0.121 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL thực hiện 3 bước: **Seq Scan** toàn bảng để lấy 200 dòng, sau đó **Sort** theo buy_turn DESC bằng thuật toán **top-N heapsort** (chỉ giữ 20 phần tử lớn nhất trong bộ nhớ thay vì sắp xếp toàn bộ), cuối cùng **Limit** cắt lấy 20 dòng đầu. Sort Method top-N heapsort chỉ tốn 37kB bộ nhớ — rất hiệu quả cho bài toán lấy top N.
 - Thời gian thực thi: Planning Time: 0.122 ms — nhanh, toàn bộ dữ liệu nằm trong bộ nhớ đệm (shared hit=8), không cần đọc từ disk.
 - Đánh giá: Câu truy vấn hoạt động hiệu quả ở quy mô hiện tại. Khi dữ liệu tăng lên lớn, có thể tạo index trên cột buy_turn để PostgreSQL chuyển sang **Index Scan** thay vì Seq Scan + Sort, tránh phải đọc và sắp xếp toàn bộ bảng mỗi lần truy vấn.

4. Yêu cầu nghiệp vụ : Tìm các lượt đánh giá 5 sao của sản phẩm “Enormous Marble Watch” có product_id = 29

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM reviews
WHERE product_id = 29 AND rating = 5;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Seq Scan on reviews (cost=0.00..22.75 rows=1 width=68) (actual time=0.020..0.023 rows=1.00 loops=1)
2	Filter: ((product_id = 29) AND (rating = 5))
3	Rows Removed by Filter: 30
4	Buffers: shared hit=1
5	Planning Time: 0.090 ms
6	Execution Time: 0.037 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng **Seq Scan** toàn bảng reviews, đọc 31 dòng, loại 30 dòng qua Filter, trả về 1 dòng khớp. Đáng chú ý shared read=1 — dữ liệu phải đọc từ **disk** thay vì RAM đệm như lần trước.
 - Thời gian thực thi: Planning Time: **2.076 ms** | Execution Time: **1.250 ms** — chậm hơn đáng kể so với lần chạy trước (0.032 ms) do buffer cache bị xóa, buộc PostgreSQL đọc lại từ disk (read=3 trong planning).
 - Đánh giá: Hiệu năng phụ thuộc nhiều vào trạng thái buffer cache — khi cache nguội, thời gian tăng gấp ~39 lần. Nên tạo index phức hợp (product_id, rating) để giảm thiểu ảnh hưởng của cache miss trong môi trường thực tế.

5. Yêu cầu nghiệp vụ : Tìm địa chỉ giao hàng mặc định của khách hàng có user_id = 5

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM addresses
WHERE user_id = 5 AND status = 'default';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Seq Scan on addresses (cost=0.00..13.73 rows=1 width=76) (actual time=0.019..0.072 rows=1.00 loops=1)
2	Filter: ((user_id = 5) AND (status = 'default';address_status_enum))
3	Rows Removed by Filter: 448
4	Buffers: shared hit=7
5	Planning Time: 0.089 ms
6	Execution Time: 0.084 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng **Seq Scan** toàn bảng addresses, đọc toàn bộ 449 dòng, loại 448 dòng qua Filter, chỉ trả về 1 dòng khớp — tỉ lệ lọc rất cao (99.8%).
 - Thời gian thực thi: Planning Time: **0.969 ms** | Execution Time: **0.117 ms** — chấp nhận được ở quy mô hiện tại.
 - Đánh giá: Tỉ lệ loại bỏ 448/449 dòng cho thấy Seq Scan rất lãng phí. Đây là truy vấn thường xuyên trong luồng checkout, nên tạo index phức hợp (user_id, status) để PostgreSQL chuyển sang **Index Scan**, tránh quét toàn bảng khi số địa chỉ tăng lên.

6. Yêu cầu nghiệp vụ : Thống kê tổng số đơn hàng của từng khách hàng trong hệ thống.

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT user_id, COUNT(*) FROM orders
GROUP BY user_id;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	HashAggregate (cost=12.00..14.04 rows=204 width=16) (actual time=0.128..0.151 rows=204.00 loops=1)
2	Group Key: user_id
3	Batches: 1 Memory Usage: 40kB
4	Buffers: shared hit=6
5	-> Seq Scan on orders (cost=0.00..10.00 rows=400 width=8) (actual time=0.015..0.045 rows=400.00 loops=1)
6	Buffers: shared hit=6
7	Planning Time: 0.097 ms
8	Execution Time: 0.188 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL thực hiện **Seq Scan** toàn bảng orders (400 dòng), sau đó **HashAggregate** gom nhóm theo user_id trong RAM (40kB) để tính COUNT — không có dòng nào bị loại vì đây là tổng hợp toàn bộ dữ liệu.
 - Thời gian thực thi: Planning Time: **0.404 ms** | Execution Time: **3.381 ms** — phần lớn thời gian do read=4 phải đọc từ disk, dữ liệu chưa được cache.
 - Đánh giá: Seq Scan là hợp lý vì truy vấn cần duyệt toàn bộ bảng để tổng hợp. Khi dữ liệu tăng lên, có thể cân nhắc tạo index trên user_id để hỗ trợ **Index Only Scan**, giảm lượng dữ liệu cần đọc từ disk.

7. Yêu cầu nghiệp vụ : Lấy danh sách tên sản phẩm và số lượng trong giỏ hàng của khách hàng có cart_id = 5.

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT p.product_name, ci.quantity FROM cart_items ci
JOIN products p ON ci.product_id = p.product_id
WHERE ci.cart_id = 5;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Hash Join (cost=12.55..23.57 rows=2 width=26) (actual time=0.086..0.105 rows=1.00 loops=1)
2	Hash Cond: (p.product_id = ci.product_id)
3	Buffers: shared hit=13
4	-> Seq Scan on products p (cost=0.00..10.00 rows=200 width=30) (actual time=0.010..0.026 rows=200.00 loops=1)
5	Buffers: shared hit=8
6	-> Hash (cost=12.53..12.53 rows=2 width=12) (actual time=0.052..0.052 rows=1.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=5
9	-> Seq Scan on cart_items ci (cost=0.00..12.53 rows=2 width=12) (actual time=0.010..0.049 rows=1.00 loops=1)
10	Filter: (cart_id = 5)
11	Rows Removed by Filter: 601
12	Buffers: shared hit=5
13	Planning:
14	Buffers: shared hit=4
15	Planning Time: 0.235 ms
16	Execution Time: 0.128 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Hash Join** để kết hợp 2 bảng — quét toàn bộ products (200 dòng) làm bảng ngoài, đồng thời **Seq Scan** cart_items (602 dòng) lọc theo cart_id = 5 loại 601 dòng chỉ giữ 1 dòng, sau đó dựng bảng Hash để JOIN theo product_id.
 - Thời gian thực thi: Planning Time: **0.270 ms** | Execution Time: **0.135 ms** — nhanh, toàn bộ dữ liệu đọc từ RAM đệm (shared hit=13).
 - Đánh giá: Seq Scan trên cart_items loại bỏ 601/602 dòng cho thấy chi phí lọc cao. Nên tạo index trên cart_items(cart_id) để PostgreSQL chuyển sang **Index Scan**, đặc biệt quan trọng khi số lượng cart items tăng lên trong môi trường thực tế.

8. Yêu cầu nghiệp vụ : Lấy toàn bộ lịch sử đơn hàng của khách hàng có user_id = 5, sắp xếp theo thời gian đặt hàng mới nhất.

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM orders
WHERE user_id = 5
ORDER BY created_at DESC;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Sort (cost=8.86..8.86 rows=2 width=104) (actual time=0.043..0.044 rows=2.00 loops=1)
2	Sort Key: created_at DESC
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=4
5	-> Bitmap Heap Scan on orders (cost=4.29..8.85 rows=2 width=104) (actual time=0.034..0.036 rows=2.00 loops=1)
6	Recheck Cond: (user_id = 5)
7	Heap Blocks: exact=2
8	Buffers: shared hit=4
9	-> Bitmap Index Scan on idx_orders_user_date (cost=0.00..4.29 rows=2 width=0) (actual time=0.021..0.021 rows=2.00 loops=1)
10	Index Cond: (user_id = 5)
11	Index Searches: 1
12	Buffers: shared hit=2
13	Planning Time: 0.112 ms
14	Execution Time: 0.068 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Bitmap Index Scan** trên index phức hợp idx_orders_user_date để lọc theo user_id = 5, sau đó **Bitmap Heap Scan** lấy 2 dòng khớp, cuối cùng **Sort** bằng quicksort trong RAM (25kB) theo created_at DESC.
 - Thời gian thực thi: Planning Time: **5.640 ms** | Execution Time: **3.296 ms** — chậm hơn bình thường do read=4 trong Heap Scan và read=7 trong Planning, cho thấy dữ liệu chưa được cache, phải đọc từ disk.
 - Đánh giá: Index idx_orders_user_date đã được tận dụng hiệu quả, không xảy ra Seq Scan. Thời gian cao chủ yếu do buffer cache ngội (dirty=2). Ở các lần truy vấn tiếp theo khi dữ liệu đã vào cache, hiệu năng sẽ cải thiện đáng kể.

9. Yêu cầu nghiệp vụ : Lấy danh sách đơn hàng được tạo trong khoảng thời gian từ 2024-01-01 đến 2024-12-31.

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM orders
WHERE created_at BETWEEN '2024-01-01' AND
'2024-12-31';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Seq Scan on orders (cost=0.00..12.00 rows=1 width=104) (actual time=0.070..0.070 rows=0.00 loops=1)
2	Filter: ((created_at >= '2024-01-01 00:00:00':timestamp without time zone) AND (created_at <= '2024-12-31 00:00:00':timestamp without time zone))
3	Rows Removed by Filter: 400
4	Buffers: shared hit=6
5	Planning Time: 0.098 ms
6	Execution Time: 0.086 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Seq Scan** toàn bảng orders (400 dòng), lọc theo khoảng thời gian BETWEEN nhưng loại bỏ toàn bộ 400 dòng, trả về 0 kết quả — do dữ liệu thực tế không có đơn hàng nào trong năm 2024.

- Thời gian thực thi Planning Time: Planning Time: **0.121 ms** | Execution Time: **0.079 ms** — rất nhanh, toàn bộ đọc từ RAM đệm (shared hit=6).
- Đánh giá : Truy vấn trả về 0 kết quả do khoảng thời gian không khớp với dữ liệu hiện có. Để tối ưu cho môi trường thực tế, nên tạo index trên created_at để PostgreSQL chuyển sang **Index Scan**, tránh Seq Scan toàn bảng khi lọc theo khoảng thời gian trên dữ liệu lớn.

10.Yêu cầu nghiệp vụ : Lấy danh sách tên sản phẩm, số lượng và đơn giá của các mặt hàng trong đơn hàng có order_id = 5.

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT p.product_name, od.quantity, od.price
FROM order_details od
JOIN products p ON od.product_id = p.product_id
WHERE od.order_id = 5;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	AZ QUERY PLAN
1	Hash Join (cost=18.91..30.19 rows=3 width=31) (actual time=0.105..0.125 rows=3.00 loops=1)
2	Hash Cond: (p.product_id = od.product_id)
3	Buffers: shared hit=17
4	-> Seq Scan on products p (cost=0.00..10.00 rows=200 width=30) (actual time=0.011..0.029 rows=200.00 loops=1)
5	Buffers: shared hit=8
6	-> Hash (cost=18.88..18.88 rows=3 width=17) (actual time=0.069..0.069 rows=3.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=9
9	-> Seq Scan on order_details od (cost=0.00..18.88 rows=3 width=17) (actual time=0.010..0.066 rows=3.00 loops=1)
10	Filter: (order_id = 5)
11	Rows Removed by Filter: 787
12	Buffers: shared hit=9
13	Planning:
14	Buffers: shared hit=4
15	Planning Time: 0.234 ms
16	Execution Time: 0.158 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: PostgreSQL sử dụng **Hash Join** — Seq Scan toàn bảng order_details (790 dòng) lọc theo order_id = 5 loại 787 dòng giữ lại 3, dựng bảng Hash rồi JOIN với Seq Scan toàn

bảng products. Toàn bộ order_details phải đọc từ disk (shared read=9).

- Thời gian thực thi: Planning Time: **3.204 ms** | Execution Time: **2.149 ms** — chậm do buffer cache nguội, order_details phải đọc hoàn toàn từ disk.
- Đánh giá: Seq Scan loại bỏ 787/790 dòng cho thấy lãng phí lớn. Nên tạo index trên order_details(order_id) để PostgreSQL chuyển sang **Index Scan**, đây là truy vấn cốt lõi hiển thị chi tiết đơn hàng nên cần tối ưu ưu tiên cao.

3.3.2. Phần truy vấn của Sinh viên 2: Nguyễn Bá Long – 202416970

1. Yêu cầu nghiệp vụ : Liệt kê đơn hàng bị hủy

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM orders WHERE status =
'cancelled';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

QUERY PLAN		text	
1	Seq Scan on orders (cost=0.00..11.00 rows=86 width=104) (actual time=0.068..0.201 rows=86.00 loops=1)		
2	Filter: (status = 'cancelled'::order_status_enum)		
3	Rows Removed by Filter: 314		
4	Buffers: shared hit=6		
5	Planning Time: 0.381 ms		
6	Execution Time: 0.256 ms		

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự toàn bảng (Seq Scan) để lọc ra 86 dòng thỏa mãn điều kiện status = 'cancelled'.
 - Thời gian thực thi: 0.381 ms | Execution Time: 0.256 ms (đọc hoàn toàn từ RAM đệm shared hit=6).
 - Đánh giá: Do quy mô bảng thử nghiệm nhỏ (400 dòng), PostgreSQL tự động chọn quét tuần tự (Seq Scan) thay vì quét chỉ mục để đạt tốc độ tối ưu nhất (0.256 ms). Tuy nhiên, khi quy mô dữ liệu đơn hàng tăng lên thực tế hàng triệu bản ghi, việc thiết lập

chỉ mục (Index) cho trường status sẽ là bắt buộc để duy trì hiệu năng và tránh nghẽn hệ thống.

2. Yêu cầu nghiệp vụ : Kiểm tra phương thức thanh toán của một đơn hàng

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT payment_method FROM payments
WHERE order_id = 3;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN text
1	Seq Scan on payments (cost=0.00..9.00 rows=1 width=4) (actual time=0.027..0.071 rows=1.00 loop...
2	Filter: (order_id = 3)
3	Rows Removed by Filter: 399
4	Buffers: shared hit=4
5	Planning Time: 0.127 ms
6	Execution Time: 0.097 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) trên toàn bộ bảng payments kết hợp với bộ lọc (Filter) để tìm ra thông tin thanh toán của đơn hàng có order_id = 3.
 - Thời gian thực thi: Planning Time: 0.127 ms | Execution Time: 0.097 ms (đọc dữ liệu từ RAM đệm shared hit=4).
 - Đánh giá: Hệ thống phải thực hiện quét toàn bảng (Seq Scan) do cột order_id chưa có chỉ mục. Tuy nhiên, nhờ quy mô dữ liệu bảng payments hiện tại đang nhỏ, tốc độ truy vấn phản hồi vẫn cực kỳ nhanh (0.097 ms), đáp ứng tức thời yêu cầu tra cứu phương thức thanh toán của một đơn hàng cụ thể.

3. Yêu cầu nghiệp vụ : Tổng tiền đã thanh toán thành công của 1 user

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT SUM(total_amount) FROM orders
WHERE user_id = 10 AND status = 'delivered';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN
	text
1	Aggregate (cost=8.30..8.31 rows=1 width=32) (actual time=0.028..0.029 rows=1.00 loops=1)
2	Buffers: shared hit=2
3	-> Index Scan using idx_orders_user_date on orders (cost=0.27..8.29 rows=1 width=6) (actual time=0.024..0.024 rows=0.00 loops=1)
4	Index Cond: (user_id = 10)
5	Filter: (status = 'delivered'::order_status_enum)
6	Index Searches: 1
7	Buffers: shared hit=2
8	Planning Time: 0.279 ms
9	Execution Time: 0.091 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng phương thức quét chỉ mục (Index Scan using idx_orders_user_date) để tra cứu nhanh mã khách hàng (user_id = 10). Tiếp theo, hệ thống sử dụng bộ lọc (Filter: status = 'delivered') để lấy ra các đơn hàng thành công, và cuối cùng dùng hàm tổng hợp (Aggregate) để cộng tổng số tiền.
 - Thời gian thực thi: Planning Time: 0.279 ms | Execution Time: 0.091 ms (dữ liệu tính toán được đọc hoàn toàn từ bộ nhớ RAM đệm với thông số shared hit=2).
 - Đánh giá: Truy vấn đạt tốc độ phản hồi cực nhanh (0.091 ms) nhờ tận dụng hiệu quả chỉ mục idx_orders_user_date để truy cập trực tiếp dữ liệu theo user_id. Hệ thống xử lý lọc và tính toán ổn định, đảm bảo tốt hiệu năng cho nghiệp vụ kiểm tra thanh toán cá nhân.

4. Yêu cầu nghiệp vụ : Các đơn hàng đang được shipper vận chuyển

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM deliveries
WHERE delivery_status = 'shipping';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN
	text
1	Seq Scan on deliveries (cost=0.00..7.04 rows=78 width=84) (actual time=0.073..0.557 rows=78.00 loops=1)
2	Filter: (delivery_status = 'shipping'::delivery_status_enum)
3	Rows Removed by Filter: 165
4	Buffers: shared hit=4
5	Planning:
6	Buffers: shared hit=91 dirtied=3
7	Planning Time: 12.930 ms
8	Execution Time: 0.629 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) trên toàn bộ bảng deliveries để đọc và lọc ra (Filter) các đơn hàng đang có trạng thái là 'shipping'.
 - Thời gian thực thi: Planning Time: 12.930 ms | Execution Time: 0.629 ms (đọc hoàn toàn từ RAM đệm shared hit=4).
 - Đánh giá: Hệ thống thực hiện quét tuần tự (Seq Scan) do cột delivery_status chưa có chỉ mục. Tuy nhiên, tốc độ phản hồi (0.629 ms) vẫn đảm bảo nhanh chóng và đáp ứng tốt nhu cầu nghiệp vụ tra cứu đơn hàng đang vận chuyển với quy mô dữ liệu hiện tại.

5. Yêu cầu nghiệp vụ : Liệt kê danh sách shippers của một đơn vị vận chuyển (carrier)

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM shippers
WHERE carrier_id = 2;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN
	text
1	Seq Scan on shippers (cost=0.00..11.12 rows=1 width=860) (actual time=0.021..0.025 rows=12.00 loops=1)
2	Filter: (carrier_id = 2)
3	Rows Removed by Filter: 38
4	Buffers: shared hit=1
5	Planning Time: 0.118 ms
6	Execution Time: 0.043 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) toàn bộ bảng shippers kết hợp bộ lọc (Filter) để lấy các dòng có carrier_id = 2.
 - Thời gian thực thi: Planning Time: 0.118 ms | Execution Time: 0.043 ms (đọc hoàn toàn từ RAM đệm shared hit=1).
 - Đánh giá: Tốc độ truy vấn hiện tại rất nhanh (0.043 ms) do bảng shippers có kích thước nhỏ. Tuy nhiên, CSDL vẫn đang phải quét toàn bộ (Seq Scan). Khi dữ liệu nhân sự tăng lên, nhóm đề xuất tạo chỉ mục bằng lệnh: CREATE INDEX idx_shippers_carrier ON

shippers(carrier_id); để chuyển sang quét Index Scan, duy trì hiệu năng tốt nhất.

6. Yêu cầu nghiệp vụ : Lấy danh sách đơn hàng cần giao trong ngày

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM deliveries
WHERE estimated_delivery_date = CURRENT_DATE;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN
	text
1	Seq Scan on deliveries (cost=0.00..7.64 rows=1 width=84) (actual time=0.313..0.313 rows=0.00 loops=1)
2	Filter: (estimated_delivery_date = CURRENT_DATE)
3	Rows Removed by Filter: 243
4	Buffers: shared hit=4
5	Planning:
6	Buffers: shared hit=6
7	Planning Time: 3.642 ms
8	Execution Time: 0.372 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) toàn bộ bảng deliveries kết hợp với bộ lọc (Filter) để xác định các đơn hàng có ngày dự kiến giao (estimated_delivery_date) trùng với ngày hiện tại (CURRENT_DATE).
 - Thời gian thực thi: Planning Time: 3.642 ms | Execution Time: 0.372 ms (đọc dữ liệu từ RAM đệm shared hit=4).
 - Đánh giá: Hệ thống thực hiện quét toàn bảng (Seq Scan) do cột ngày tháng chưa có chỉ mục. Tuy nhiên, tốc độ truy vấn phản hồi rất nhanh (0.372 ms) nhờ quy mô dữ liệu thử nghiệm nhỏ, đáp ứng tốt yêu cầu tra cứu đơn hàng cần giao trong ngày.

7. Yêu cầu nghiệp vụ : Tìm shipper đang rảnh (status = 'active')

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT * FROM shippers
WHERE status = 'active';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN	
	text	
1	Seq Scan on shippers (cost=0.00..11.12 rows=1 width=860) (actual time=0.096..0.101 rows=50.00 loops=1)	
2	Filter: (status = 'active'::shipper_status_enum)	
3	Buffers: shared hit=1	
4	Planning Time: 0.292 ms	
5	Execution Time: 0.152 ms	

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) toàn bộ bảng shippers kết hợp với bộ lọc (Filter) để xác định các shipper có trạng thái hoạt động là 'active'.
 - Thời gian thực thi: Planning Time: 0.292 ms | Execution Time: 0.152 ms (đọc dữ liệu từ RAM đệm shared hit=1).
 - Đánh giá: Hệ thống thực hiện quét toàn bảng (Seq Scan) do cột status chưa có chỉ mục. Tuy nhiên, tốc độ truy vấn phản hồi rất nhanh (0.152 ms), đảm bảo tốt hiệu năng cho nghiệp vụ tra cứu nhanh danh sách shipper đang sẵn sàng nhận đơn.

8. Yêu cầu nghiệp vụ : Tổng doanh thu hệ thống

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT SUM(total_amount) FROM orders
WHERE status = 'delivered';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN	
	text	
1	Aggregate (cost=11.20..11.21 rows=1 width=32) (actual time=3.137..3.138 rows=1.00 loops=1)	
2	Buffers: shared hit=6	
3	-> Seq Scan on orders (cost=0.00..11.00 rows=79 width=6) (actual time=0.102..0.275 rows=79.00 loops=1)	
4	Filter: (status = 'delivered'::order_status_enum)	
5	Rows Removed by Filter: 321	
6	Buffers: shared hit=6	
7	Planning Time: 0.332 ms	
8	Execution Time: 3.235 ms	

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) toàn bộ bảng orders kết hợp bộ lọc (Filter) để chọn các đơn hàng đã giao thành công ('delivered'), sau đó sử dụng hàm tổng hợp (Aggregate) để tính tổng doanh thu

- Thời gian thực thi: Planning Time: 0.332 ms | Execution Time: 3.235 ms (đọc dữ liệu từ RAM đệm shared hit=6).
- Đánh giá: Hệ thống thực hiện quét toàn bảng (Seq Scan) để lọc theo trạng thái đơn hàng. Với quy mô dữ liệu thử nghiệm, thời gian xử lý vẫn nằm trong ngưỡng cho phép, đảm bảo cung cấp kết quả tổng doanh thu chính xác cho nhu cầu phân tích báo cáo của hệ thống.

9. Yêu cầu nghiệp vụ : Top 10 sản phẩm bán chạy nhất tháng

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT product_id, SUM(quantity) FROM order_details
GROUP BY product_id
ORDER BY SUM(quantity) DESC LIMIT 10;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN text	
1	Limit (cost=27.11..27.13 rows=10 width=16) (actual time=3.128..3.133 rows=10.00 loops=1)	
2	Buffers: shared hit=12	
3	-> Sort (cost=27.11..27.60 rows=198 width=16) (actual time=2.784..2.787 rows=10.00 loops=1)	
4	Sort Key: (sum(quantity)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=12	
7	-> HashAggregate (cost=20.85..22.83 rows=198 width=16) (actual time=1.599..1.657 rows=198.00 loops=1)	
8	Group Key: product_id	
9	Batches: 1 Memory Usage: 32kB	
10	Buffers: shared hit=9	
11	-> Seq Scan on order_details (cost=0.00..16.90 rows=790 width=12) (actual time=0.139..1.136 rows=790.00 loops=1)	
12	Buffers: shared hit=9	
13	Planning:	
14	Buffers: shared hit=32 dirtied=1	
15	Planning Time: 10.346 ms	
16	Execution Time: 4.186 ms	

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự (Seq Scan) toàn bộ bảng order_details, kết hợp với HashAggregate để gom nhóm theo product_id, sau đó dùng thuật toán Top-N Heapsort để sắp xếp và lọc ra 10 sản phẩm có tổng số lượng bán cao nhất.
 - Thời gian thực thi: Planning Time: 10.346 ms | Execution Time: 4.186 ms (đọc dữ liệu từ RAM đệm shared hit=12).
 - Đánh giá: Hệ thống sử dụng phương pháp HashAggregate và Top-N Heapsort hiệu quả để xử lý việc gom nhóm và sắp xếp dữ liệu.

Truy vấn đạt tốc độ phản hồi nhanh, đáp ứng tốt nhu cầu phân tích báo cáo các sản phẩm bán chạy theo tháng.

10. Yêu cầu nghiệp vụ : Cửa hàng có doanh thu cao nhất

- Mã SQL thực thi:

```
-- EXPLAIN ANALYZE
SELECT store_id, SUM(o.total_amount) FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN products p ON od.product_id = p.product_id
GROUP BY store_id
ORDER BY SUM(o.total_amount) DESC LIMIT 1;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

	QUERY PLAN	
	text	
1	Limit (cost=78.60..78.60 rows=1 width=40) (actual time=0.709..0.710 rows=1.00 loops=1)	
2	Buffers: shared hit=48	
3	-> Sort (cost=78.60..78.74 rows=59 width=40) (actual time=0.707..0.708 rows=1.00 loops=1)	
4	Sort Key: (sum(o.total_amount)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=48	
7	-> HashAggregate (cost=77.56..78.30 rows=59 width=40) (actual time=0.676..0.689 rows=59.00 loops=1)	
8	Group Key: p.store_id	
9	Batches: 1 Memory Usage: 56kB	
10	Buffers: shared hit=48	
11	-> Hash Join (cost=52.50..73.61 rows=790 width=14) (actual time=0.214..0.513 rows=790.00 loops=1)	
12	Hash Cond: (od.product_id = p.product_id)	
13	Buffers: shared hit=48	
14	-> Hash Join (cost=15.00..34.00 rows=790 width=14) (actual time=0.126..0.323 rows=790.00 loops=1)	
15	Hash Cond: (od.order_id = o.order_id)	
16	Buffers: shared hit=15	
17	-> Seq Scan on order_details od (cost=0.00..16.90 rows=790 width=16) (actual time=0.020..0.062 rows=790.00 loops=1)	
18	Buffers: shared hit=9	
19	-> Hash (cost=10.00..10.00 rows=400 width=14) (actual time=0.096..0.097 rows=400.00 loops=1)	
20	Buckets: 1024 Batches: 1 Memory Usage: 27kB	
21	Buffers: shared hit=6	
22	-> Seq Scan on orders o (cost=0.00..10.00 rows=400 width=14) (actual time=0.009..0.055 rows=400.00 loops=1)	
23	Buffers: shared hit=6	
24	-> Hash (cost=35.00..35.00 rows=200 width=16) (actual time=0.079..0.080 rows=200.00 loops=1)	
25	Buckets: 1024 Batches: 1 Memory Usage: 18kB	
26	Buffers: shared hit=33	
27	-> Seq Scan on products p (cost=0.00..35.00 rows=200 width=16) (actual time=0.010..0.059 rows=200.00 loops=1)	
28	Buffers: shared hit=33	
29	Planning:	
30	Buffers: shared hit=10	
31	Planning Time: 1.225 ms	
32	Execution Time: 0.782 ms	

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Hệ thống sử dụng phép toán Hash Join để kết nối dữ liệu từ ba bảng orders, order_details và products. Sau đó, tiến hành gom nhóm (HashAggregate) theo store_id và dùng thuật toán top-N heapsort kết hợp với Limit để lấy ra chính xác 1 cửa hàng có tổng doanh thu cao nhất.
 - Thời gian thực thi Planning Time: 1.225 ms | Execution Time: 0.782 ms (đọc dữ liệu từ RAM đệm shared hit=48).
 - Đánh giá : Dù phải xử lý kết nối ba bảng, truy vấn vẫn đạt tốc độ thực thi rất tốt (0.782 ms). Nhờ việc bổ sung giới hạn kết quả (LIMIT 1) kết hợp với thuật toán top-N heapsort, hệ thống không cần phải sắp xếp toàn bộ dữ liệu mà chỉ giữ lại phần tử lớn nhất, đáp ứng hoàn hảo và tối ưu yêu cầu tìm kiếm top 1 cửa hàng.

3.3.3. Phần truy vấn của Sinh viên 3: Nguyễn Thị Thanh Trúc – 202417052

1. **Yêu cầu nghiệp vụ:** Lọc các sản phẩm đang bán, còn hàng trong khoảng giá từ 500,000 đến 2,000,000 VND và sắp xếp từ sản phẩm mới đăng nhất đến cũ nhất.

- Mã SQL thực thi:

```
-- explain analyze
select product_id, product_name, price, created_at
from products
where (discontinued = false) and (quantity > 5)
and (price between 500000.00 and 2000000.00)
order by created_at desc;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN
Sort (cost=17.11..17.18 rows=26 width=45) (actual time=0.082..0.084 rows=26.00 loops=1)
Sort Key: created_at DESC
Sort Method: quicksort Memory: 26kB
Buffers: shared hit=13
-> Seq Scan on products (cost=0.00..16.50 rows=26 width=45) (actual time=0.016..0.071 rows=26.00 loops=1)
Filter: ((NOT discontinued) AND (quantity > 5) AND (price >= 500000.00) AND (price <= 2000000.00))
Rows Removed by Filter: 174
Buffers: shared hit=13
Planning Time: 0.111 ms
Execution Time: 0.108 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Sử dụng quét tuần tự toàn bảng (**Seq Scan**) để lọc ra 26 dòng thỏa mãn, sau đó sắp xếp bằng thuật toán **quicksort** trong RAM theo thứ tự `created_at` DESC.
 - Thời gian thực thi: Planning Time: **0.111 ms** | Execution Time: **0.108 ms** (đọc hoàn toàn từ RAM đệm shared hit=13).
 - Đánh giá: Do quy mô bảng thử nghiệm nhỏ (200 dòng), PostgreSQL tự động chọn quét tuần tự (Seq Scan) thay vì quét chỉ mục để đạt tốc độ tối ưu nhất (0.108 ms). Chỉ mục phức hợp đã cài đặt sẵn sẽ tự động phát huy tác dụng khi quy mô dữ liệu tăng lên thực tế.

2. Yêu cầu nghiệp vụ: Tính điểm đánh giá trung bình của từng cửa hàng dựa trên các lượt đánh giá sản phẩm của khách hàng, nhằm phục vụ việc theo dõi và hiển thị chất lượng dịch vụ của các cửa hàng trên sàn TMĐT.

- Mã SQL thực thi:

```
-- explain analyze
select store_id, avg(rating)
from reviews r join products p
on r.product_id = p.product_id
group by store_id;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

HashAggregate (cost=42.53..43.26 rows=59 width=40) (actual time=0.425..0.447 rows=23.00 loops=1)
Group Key: p.store_id
Batches: 1 Memory Usage: 32kB
Buffers: shared hit=14
-> Hash Join (cost=17.50..38.28 rows=850 width=12) (actual time=0.369..0.396 rows=31.00 loops=1)
Hash Cond: (r.product_id = p.product_id)
Buffers: shared hit=14
-> Seq Scan on reviews r (cost=0.00..18.50 rows=850 width=12) (actual time=0.047..0.053 rows=31.00 loops=1)
Buffers: shared hit=1
-> Hash (cost=15.00..15.00 rows=200 width=16) (actual time=0.281..0.282 rows=200.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 18kB
Buffers: shared hit=13
-> Seq Scan on products p (cost=0.00..15.00 rows=200 width=16) (actual time=0.036..0.204 rows=200.00 loops=1)
Buffers: shared hit=13
Planning Time: 0.419 ms
Execution Time: 0.566 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu: Hệ thống thực hiện quét tuần tự (Seq Scan) trên cả hai bảng products và reviews, kết hợp liên kết băm (Hash Join) theo product_id. Kết quả sau đó được gom nhóm bằng bảng băm (HashAggregate) theo store_id để tính điểm đánh giá trung bình.
 - Thời gian thực thi: Planning Time: 0.419 ms | Execution Time: 0.566 ms (đọc hoàn toàn từ RAM đệm shared hit=14).
 - Đánh giá: Do quy mô dữ liệu của hai bảng rất nhỏ, việc quét tuần tự kết hợp liên kết băm đạt hiệu năng tối ưu nhất mà không cần chỉ mục. Khi dữ liệu phình to trong thực tế, cần đánh thêm chỉ mục trên trường liên kết reviews(product_id) để tránh việc quét toàn bảng gây chậm hệ thống.

3. Yêu cầu nghiệp vụ: Lọc ra các gian hàng mới đăng ký hoạt động trong giai đoạn từ ngày 01/05/2026 đến hết ngày 31/05/2026 để gửi thư chào mừng của hệ thống.

- Mã SQL thực thi:

```
-- explain analyze
select store_id, store_name, created_at
from stores
where created_at between '2026-05-01 00:00:00'
and '2026-05-31 23:59:59';
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN
Seq Scan on stores (cost=0.00..2.90 rows=1 width=46) (actual time=0.020..0.021 rows=1.00 loops=1)
Filter: ((created_at >= '2026-05-01 00:00:00':timestamp without time zone) AND (created_at <= '2026-05-31 23:59:59':timestamp without time zone))
Rows Removed by Filter: 60
Buffers: shared hit=2
Planning Time: 0.061 ms
Execution Time: 0.032 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - **Phương thức quét dữ liệu:** Sử dụng quét tuần tự toàn bảng (Seq Scan on stores) để duyệt toàn bộ 61 bản ghi, kết hợp bộ lọc (Filter) lọc ra đúng 1 cửa hàng đăng ký mới thỏa mãn khoảng thời gian trong tháng 05/2026.

- **Thời gian thực thi:** Planning Time: **0.061 ms** | Execution Time: **0.032 ms** (đọc hoàn toàn từ RAM đệm shared hit=2).
- **Đánh giá:** Do quy mô dữ liệu của bảng stores rất nhỏ (chỉ 61 dòng, đọc vón vẹn 2 blocks dữ liệu), PostgreSQL tự động lựa chọn quét tuần tự (Seq Scan) thay vì dùng chỉ mục để đạt tốc độ xử lý nhanh nhất (0.032 ms). Để tối ưu hơn khi quy mô của hàng đạt hàng trăm nghìn dòng trong thực tế, ta có thể đánh thêm chỉ mục trên cột created_at.

4. Yêu cầu nghiệp vụ: Tính toán tỷ lệ phần trăm đơn hàng bị hủy trên tổng số lượng đơn hàng phát sinh của sản để phòng ngừa rủi ro.

- Mã SQL thực thi:

```
-- explain analyze
select avg(case
when status = 'cancelled' then 1.0
else 0.0 end) * 100 as cancellation_rate_percent
from orders;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

A-Z QUERY PLAN
Aggregate (cost=12.00..12.01 rows=1 width=32) (actual time=0.104..0.104 rows=1.00 loops=1)
Buffers: shared hit=6
-> Seq Scan on orders (cost=0.00..10.00 rows=400 width=4) (actual time=0.012..0.042 rows=400.00 loops=1)
Buffers: shared hit=6
Planning Time: 0.064 ms
Execution Time: 0.122 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - **Phương thức quét dữ liệu:** Sử dụng quét tuần tự toàn bảng (**Seq Scan on orders**) để duyệt qua 400 bản ghi của bảng đơn hàng, sau đó thực hiện tính toán gộp nhóm tích hợp biểu thức điều kiện (**Aggregate** dùng CASE WHEN) để tính ra tỉ lệ đơn bị hủy.
 - **Thời gian thực thi:** Planning Time: **0.064 ms** | Execution Time: **0.122 ms** (đọc hoàn toàn từ RAM đệm shared hit=6).
 - **Đánh giá:** Do quy mô dữ liệu của bảng orders thử nghiệm nhỏ (chỉ có 400 dòng, đọc vón vẹn 6 khối dữ liệu từ bộ nhớ đệm), PostgreSQL tự động chọn quét tuần tự thay vì dùng chỉ mục nhằm đạt tốc độ xử lý nhanh nhất (0.122 ms). Dữ liệu được xử lý 100% trên bộ đệm RAM giúp tốc độ phản hồi ở mức tức thời.

5. Yêu cầu nghiệp vụ: Chuyển trạng thái deleted = true khi khách hàng yêu cầu đóng/xóa tài khoản để bảo toàn lịch sử dữ liệu của sản.

- Mã SQL thực thi:

```
-- explain analyze
update users set deleted = true
where user_id = ???;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

A-Z QUERY PLAN
Update on users (cost=0.15..8.17 rows=0 width=0) (actual time=3.150..3.151 rows=0.00 loops=1)
Buffers: shared hit=20 read=8 dirtied=6
-> Index Scan using users_pkey on users (cost=0.15..8.17 rows=1 width=7) (actual time=0.015..0.018 row
Index Cond: (user_id = 10)
Index Searches: 1
Buffers: shared hit=2
Planning Time: 0.088 ms
Execution Time: 3.198 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu:** Sử dụng quét chỉ mục (**Index Scan** dựa trên khóa chính users_pkey) để tìm kiếm tức thời tài khoản của khách hàng có user_id = 10 trong bảng users, sau đó thực hiện ghi đè cập nhật xóa mềm (**Update on users**).
 - Thời gian thực thi:** Planning Time: **0.088 ms** | Execution Time: **3.198 ms** (thao tác đọc ghi trên bộ đệm với shared hit=20, đọc từ đĩa cứng read=8 và đánh dấu ghi bẩn dirtied=6).
 - Đánh giá:** Hệ thống tối ưu hóa vượt trội nhờ việc sử dụng chỉ mục khóa chính có sẵn, giúp việc định vị tài khoản người dùng chỉ mất vón vẹn 0.018 ms. Tổng thời gian thực thi ghi dữ liệu mất 3.198 ms do có phát sinh I/O đọc đĩa vật lý (read=8) và tạo trang ghi bẩn đệm (dirtied=6) chờ đồng bộ xuống ổ cứng. Hiệu năng đạt chuẩn an toàn cao.

6. Yêu cầu nghiệp vụ: Thay đổi trạng thái hoạt động của cửa hàng sang banned khi phát hiện vi phạm quy chế hoạt động của sản TMĐT.

- Mã SQL thực thi:

```
-- explain analyze
update stores set status = 'banned'
where store_id = ???;
```


- Kết quả thực thi EXPLAIN ANALYZE thực tế:

A-Z QUERY PLAN
Update on stores (cost=0.00..2.75 rows=0 width=0) (actual time=1.138..1.139 rows=0.00 loops=1)
Buffers: shared hit=12 dirtied=4
-> Seq Scan on stores (cost=0.00..2.75 rows=1 width=10) (actual time=0.015..0.027 rows=1.00 loops=1)
Filter: (store_id = 1)
Rows Removed by Filter: 60
Buffers: shared hit=2
Planning Time: 0.064 ms
Execution Time: 3.777 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - **Phương thức quét dữ liệu:** Sử dụng quét tuần tự (**Seq Scan on stores**) kết hợp điều kiện lọc để định vị nhanh dòng dữ liệu của cửa hàng có store_id = 1, sau đó thực hiện ghi đè cập nhật trạng thái mới (**Update on stores**).
 - **Thời gian thực thi:** Planning Time: **0.064 ms** | Execution Time: **3.777 ms** (đọc ghi dữ liệu trên RAM đệm shared hit=12 và tạo trang bản chờ đồng bộ xuống đĩa dirtied=4).
 - **Đánh giá:** Do quy mô bảng stores nhỏ (chỉ 61 dòng), PostgreSQL tự động quét tuần tự tìm kiếm dòng đích trước khi cập nhật. Tổng thời gian thực thi mất 3.777 ms (lâu hơn phép truy vấn đọc dữ liệu thông thường do phải ghi sửa đổi lên RAM đệm và tạo trang ghi bản dirtied=4 chờ đồng bộ xuống ổ cứng). Hiệu năng ghi dữ liệu như vậy là hoàn toàn đạt yêu cầu.

7. Yêu cầu nghiệp vụ: Thống kê xem tỉnh/thành phố nào đang có lượng đơn đặt hàng lớn nhất để sản lên kế hoạch mở rộng chi nhánh kho bãi hoặc liên kết thêm đơn vị vận chuyển tại khu vực đó.

- Mã SQL thực thi:

```
-- explain analyze
select a.city, count(o.order_id) as
tong_so_don_hang
from orders o join addresses a
on o.ship_address_id = a.address_id
group by a.city
order by tong_so_don_hang desc;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN
Sort (cost=31.03..31.10 rows=26 width=16) (actual time=0.383..0.386 rows=26.00 loops=1)
Sort Key: (count(o.order_id)) DESC
Sort Method: quicksort Memory: 25kB
Buffers: shared hit=13
-> HashAggregate (cost=30.16..30.42 rows=26 width=16) (actual time=0.371..0.375 rows=26.00 loops=1)
Group Key: a.city
Batches: 1 Memory Usage: 32kB
Buffers: shared hit=13
-> Hash Join (cost=17.10..28.16 rows=400 width=16) (actual time=0.149..0.276 rows=400.00 loops=1)
Hash Cond: (o.ship_address_id = a.address_id)
Buffers: shared hit=13
-> Seq Scan on orders o (cost=0.00..10.00 rows=400 width=16) (actual time=0.010..0.041 rows=400.00 loops=1)
Buffers: shared hit=6
-> Hash (cost=11.49..11.49 rows=449 width=16) (actual time=0.133..0.134 rows=449.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 32kB
Buffers: shared hit=7
-> Seq Scan on addresses a (cost=0.00..11.49 rows=449 width=16) (actual time=0.006..0.065 rows=449.00 loops=1)
Buffers: shared hit=7
Planning:
Buffers: shared hit=6
Planning Time: 0.179 ms
Execution Time: 0.424 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - **Phương thức quét dữ liệu:** Quét tuần tự (Seq Scan) trên cả hai bảng orders và addresses, thực hiện liên kết dữ liệu bằng giải thuật băm **Hash Join** qua trường khóa ngoại. Tiếp theo, hệ thống tiến hành gộp nhóm bằng **HashAggregate** theo thành phố (a.city) và sắp xếp bằng thuật toán **quicksort** trong RAM theo thứ tự giảm dần của tổng số đơn.
 - **Thời gian thực thi:** Planning Time: **0.179 ms** | Execution Time: **0.424 ms** (đọc hoàn toàn từ RAM đệm shared hit=13).
 - **Đánh giá:** Mặc dù thực hiện liên kết bảng và gộp nhóm thống kê trên quy mô dữ liệu mẫu (400 đơn hàng và 449 địa chỉ), PostgreSQL đã tính toán tối ưu bằng cách lưu dữ liệu trung gian vào RAM và sử dụng các thuật toán băm hiệu năng cao. Thời gian phản hồi cực nhanh (0.424 ms), đảm bảo tốc độ xuất báo cáo thống kê tức thời.

8. Yêu cầu nghiệp vụ: Hiển thị 20 lượt nhận xét, đánh giá sản phẩm mới nhất của khách hàng lên trang chủ.

- Mã SQL thực thi:

```
-- explain analyze
select * from reviews
order by review_date desc limit 20;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN
Limit (cost=41.12..41.17 rows=20 width=68) (actual time=0.289..0.293 rows=20.00 loops=1)
Buffers: shared read=1
-> Sort (cost=41.12..43.24 rows=850 width=68) (actual time=0.288..0.290 rows=20.00 loops=1)
Sort Key: review_date DESC
Sort Method: quicksort Memory: 32kB
Buffers: shared read=1
-> Seq Scan on reviews (cost=0.00..18.50 rows=850 width=68) (actual time=0.272..0.275 rows=31.00 loops=1)
Buffers: shared read=1
Planning:
Buffers: shared hit=27 read=1
Planning Time: 1.721 ms
Execution Time: 0.311 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu:** Sử dụng quét tuần tự toàn bảng (**Seq Scan on reviews**), tiến hành sắp xếp dữ liệu bằng thuật toán **quicksort** trong RAM (tiêu thụ 32kB bộ nhớ) theo thứ tự review_date DESC và cuối cùng áp dụng bộ lọc **Limit** để trích xuất ra đúng 20 bản ghi mới nhất.
 - Thời gian thực thi:** Planning Time: **1.721 ms** | Execution Time: **0.311 ms** (đọc trực tiếp từ đĩa cứng vật lý shared read=1).
 - Đánh giá:** Hệ thống phản hồi cực nhanh (0.311 ms) nhờ dữ liệu bảng nhỏ và thuật toán sắp xếp RAM tối ưu. Để tối ưu hóa triệt để khi bảng đánh giá tăng trưởng lớn trong thực tế, nhóm có thể tạo thêm chỉ mục B-Tree trên cột review_date DESC nhằm giúp hệ thống lấy ngay kết quả mà không cần tốn chi phí chạy thuật toán sắp xếp (Sort).

9. Yêu cầu nghiệp vụ: Đếm số lượng đơn hàng giao trễ của từng đối tác để làm tiêu chí đánh giá chất lượng KPI giao nhận.

- Mã SQL thực thi:

```
-- explain analyze
select c.carrier_name,
count(d.delivery_id) as so_don_giao_tre
from carriers c join deliveries d
on c.carrier_id = d.carrier_id
where d.delivery_status = 'delivered'
and d.delivered_at > d.estimated_delivery_date
group by c.carrier_name;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

AZ QUERY PLAN
HashAggregate (cost=18.14..18.40 rows=26 width=226) (actual time=0.090..0.092 rows=0.00 loops=1)
Group Key: c.carrier_name
Batches: 1 Memory Usage: 32kB
Buffers: shared hit=4
-> Nested Loop (cost=0.15..18.01 rows=26 width=226) (actual time=0.088..0.090 rows=0.00 loops=1)
Buffers: shared hit=4
-> Seq Scan on deliveries d (cost=0.00..7.64 rows=26 width=16) (actual time=0.088..0.088 rows=0.00 loops=1)
Filter: ((delivered_at > estimated_delivery_date) AND (delivery_status = 'delivered'::delivery_status_enum))
Rows Removed by Filter: 243
Buffers: shared hit=4
-> Memoize (cost=0.15..2.02 rows=1 width=226) (never executed)
Cache Key: d.carrier_id
Cache Mode: logical
-> Index Scan using carriers_pkey on carriers c (cost=0.14..2.01 rows=1 width=226) (never executed)
Index Cond: (carrier_id = d.carrier_id)
Index Searches: 0
Planning Time: 0.276 ms
Execution Time: 0.151 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - Phương thức quét dữ liệu:** Sử dụng quét tuần tự (**Seq Scan on deliveries**) để lọc ra các đơn trễ hạn. Do kết quả lọc trả về 0 dòng thỏa mãn, phép liên kết vòng lặp lồng (**Nested Loop**) kết hợp cơ chế lưu đệm (**Memoize**) và quét chỉ mục khóa chính (**Index Scan using carriers_pkey**) trên bảng carriers không cần phải thực thi. Cuối cùng, hệ thống gộp nhóm bằng **HashAggregate** theo tên đối tác.
 - Thời gian thực thi:** Planning Time: **0.276 ms** | Execution Time: **0.151 ms** (đọc hoàn toàn từ RAM đệm shared hit=4).
 - Đánh giá:** Hệ thống tối ưu hóa cực kỳ thông minh khi kết hợp chỉ mục khóa chính và cơ chế **Memoize**. Vì không có đơn hàng nào bị giao trễ trong tập dữ liệu thử nghiệm (rows=0.00, 243 dòng bị loại

bởi bộ lọc đĩa), PostgreSQL lập tức dừng sớm tiến trình kết nối bảng giúp tiết kiệm tối đa tài nguyên I/O. Thời gian phản hồi siêu nhanh (0.151 ms).

10. Yêu cầu nghiệp vụ: Lọc ra các gian hàng hoạt động xuất sắc có tổng doanh thu thực tế (đơn hàng đã giao thành công) đạt trên 50 triệu đồng để nâng cấp nhãn "Cửa hàng VIP".

- Mã SQL thực thi:

```
-- explain analyze
select s.store_id, s.store_name,
sum(od.total_amount) as doanh_thu
from stores s
join products p on s.store_id = p.store_id
join order_details od on p.product_id =
od.product_id
join orders o on od.order_id = o.order_id
where o.status = 'delivered'
group by s.store_id, s.store_name
having sum(od.total_amount) > 50000000.00;
```

- Kết quả thực thi EXPLAIN ANALYZE thực tế:

A-Z QUERY PLAN
HashAggregate (cost=55.76..56.66 rows=20 width=70) (actual time=0.547..0.550 rows=0.00 loops=1)
Group Key: s.store_name
Filter: (sum(od.total_amount) > 50000000.00)
Batches: 1 Memory Usage: 40kB
Rows Removed by Filter: 49
Buffers: shared hit=31
-> Hash Join (cost=32.84..54.90 rows=173 width=44) (actual time=0.196..0.467 rows=155.00 loops=1)
Hash Cond: (p.store_id = s.store_id)
Buffers: shared hit=31
-> Hash Join (cost=29.49..51.06 rows=173 width=14) (actual time=0.154..0.382 rows=155.00 loops=1)
Hash Cond: (od.product_id = p.product_id)
Buffers: shared hit=29
-> Hash Join (cost=11.99..33.09 rows=173 width=14) (actual time=0.077..0.262 rows=155.00 loops=1)
Hash Cond: (od.order_id = o.order_id)
Buffers: shared hit=16
-> Seq Scan on order_details od (cost=0.00..18.78 rows=878 width=22) (actual time=0.004..0.004 rows=878 loops=1)
Buffers: shared hit=10
-> Hash (cost=11.00..11.00 rows=79 width=8) (actual time=0.067..0.067 rows=79.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 12kB
Buffers: shared hit=6

-> Seq Scan on orders o (cost=0.00..11.00 rows=79 width=8) (actual time=0.005..0.054 rows=79)
Filter: (status = 'delivered'::order_status_enum)
Rows Removed by Filter: 326
Buffers: shared hit=6
-> Hash (cost=15.00..15.00 rows=200 width=16) (actual time=0.071..0.071 rows=200.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 18kB
Buffers: shared hit=13
-> Seq Scan on products p (cost=0.00..15.00 rows=200 width=16) (actual time=0.004..0.042 rows=200.00)
Buffers: shared hit=13
-> Hash (cost=2.60..2.60 rows=60 width=38) (actual time=0.035..0.035 rows=61.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 13kB
Buffers: shared hit=2
-> Seq Scan on stores s (cost=0.00..2.60 rows=60 width=38) (actual time=0.011..0.019 rows=61.00)
Buffers: shared hit=2
Planning:
Buffers: shared hit=17
Planning Time: 0.405 ms
Execution Time: 0.619 ms

- Phân tích hiệu năng và đánh giá tối ưu:
 - **Phương thức quét dữ liệu:** Thực hiện kết nối cùng lúc 4 bảng bằng chuỗi giải thuật băm **Hash Join** dựa trên việc quét tuần tự (Seq Scan) dữ liệu của từng bảng. Sau đó, hệ thống gộp nhóm bằng **HashAggregate** theo tên cửa hàng và lọc điều kiện doanh thu lớn hơn 50 triệu ở mệnh đề HAVING (Filter: (sum(od.total_amount) > 50000000.00)).
 - **Thời gian thực thi:** Planning Time: **0.405 ms** | Execution Time: **0.619 ms** (đọc hoàn toàn từ RAM đệm shared hit=30).
 - **Đánh giá:** Mặc dù là truy vấn rất phức tạp liên kết cùng lúc 4 bảng và tính toán gộp nhóm, PostgreSQL đã tối ưu hóa xuất sắc các liên kết nhờ giải thuật băm trên bộ nhớ đệm RAM (shared hit=30). Thời gian chạy cực nhanh (0.559 ms). Do dữ liệu thử nghiệm chưa có cửa hàng nào đạt mốc doanh thu 50 triệu nên hệ thống trả về 0 dòng kết quả (có 49 cửa hàng bị loại bỏ bởi bộ lọc Filter).

PHẦN 4: ĐÁNH GIÁ & TỔNG KẾT

1. Đánh giá kết quả đạt được

Sau toàn bộ quá trình nghiên cứu, thiết kế và triển khai thực tế, nhóm tiến hành đánh giá khách quan về kết quả đạt được của dự án bài tập lớn theo các khía cạnh ưu điểm và nhược điểm như sau:

Những ưu điểm nổi bật đạt được (Pros)

- **Cấu trúc CSDL chặt chẽ và nhất quán:** Thiết kế thành công lược đồ quan hệ đạt chuẩn chuẩn hóa 3NF trên PostgreSQL. Các thực thể cốt lõi (users, products, orders, deliveries, addresses...) được ràng buộc chặt chẽ bằng hệ thống khóa chính, khóa ngoại rõ ràng và các kiểu dữ liệu liệt kê (ENUM như payment_method_enum, order_status_enum), đảm bảo tính toàn vẹn dữ liệu và ngăn chặn hoàn toàn hiện tượng mâu thuẫn hay mất mát thông tin.
- **Khai thác hiệu quả tính năng nâng cao dưới CSDL:** Dự án không đẩy toàn bộ logic xử lý lên mã nguồn Java mà tận dụng triệt để sức mạnh của hệ quản trị PostgreSQL. Nhóm đã tự viết và áp dụng thành công bộ kích hoạt tự động (Trigger: trigger_update_stock trên order_details) để tự động kiểm tra, trừ kho và cộng dồn lượt bán (buy_turn) của sản phẩm khi phát sinh đơn hàng, giúp bảo vệ dữ liệu ở mức vật lý.
- **Tích hợp phần mềm Java và JDBC an toàn, đồng bộ:**
 - Kết nối thành công ứng dụng Desktop Swing với PostgreSQL qua JDBC sử dụng hoàn toàn PreparedStatement để phòng chống tấn công SQL Injection.
 - Lập trình cấu trúc mã nguồn theo hướng mô-đun hóa cao (tách riêng các lớp StorefrontPanel, CartPanel, SuccessReceiptPanel...) giúp ứng dụng dễ đọc và bảo trì.
 - Quản lý tiến trình chốt đơn bằng một **Database Transaction** khép kín (setAutoCommit(false)), đảm bảo tính nhất quán (ACID) của nghiệp vụ thanh toán và tự động điều phối Shipper rảnh rỗi.
 - Thiết kế lưu trữ hình ảnh thông minh: Ảnh vật lý được lưu trong thư mục cục bộ photo_for_database prj, database chỉ lưu đường dẫn tương đối dạng văn bản giúp bảo vệ hiệu năng hệ thống không bị phình to dung lượng.

- **Củng cố kiến thức thực tế:** Qua việc tự tay thiết kế bảng, viết script SQL (nạp dữ liệu mẫu bằng tools_datagen, đồng bộ sequence) và gỡ lỗi (debug) các lỗi kỹ thuật thực tế, các thành viên trong nhóm đã thực sự hiểu sâu cách vận hành của một hệ thống cơ sở dữ liệu thực tiễn thay vì chỉ học lý thuyết đơn thuần.

Những hạn chế và nhược điểm cần cải tiến (Cons)

Mặc dù đáp ứng đầy đủ yêu cầu nghiệp vụ mô phỏng, hệ thống vẫn tồn tại một số hạn chế kỹ thuật cần khắc phục trong tương lai:

- **Kiến trúc kết nối trực tiếp (2-Tier Architecture):** Ứng dụng client Java Swing tương tác trực tiếp với cơ sở dữ liệu PostgreSQL thông qua JDBC. Trong các hệ thống doanh nghiệp thực tế, điều này gây rủi ro bảo mật (vì lộ thông tin kết nối DB trực tiếp tại client) và khó mở rộng. Hệ thống cần được cải tiến lên kiến trúc 3 lớp (3-Tier) bằng cách xây dựng một lớp trung gian làm Backend Web API (như Spring Boot/NodeJS).
- **Giới hạn mạng cục bộ (Localhost Dependency):** Do kết nối trực tiếp tới cơ sở dữ liệu PostgreSQL cài đặt cục bộ tại máy cá nhân, hệ thống chưa thể vận hành đồng bộ dữ liệu giữa nhiều máy tính của khách hàng khác nhau qua mạng Internet (trừ khi cấu hình CSDL đám mây hoặc IP tĩnh).
- **Quản lý phiên làm việc tạm thời trong bộ nhớ:** Thông tin đăng nhập hiện tại (UserSession) được lưu trữ tạm thời bằng các biến static trong bộ nhớ RAM của Java. Khi tắt ứng dụng, phiên làm việc sẽ bị mất hoàn toàn và yêu cầu đăng nhập lại, chưa có cơ chế lưu trữ phiên làm việc dài hạn như Token bảo mật (JWT) hay Cookie.
- **Sự phụ thuộc vào đường dẫn hình ảnh vật lý cục bộ:** Việc tải ảnh sản phẩm lên giao diện dựa vào đường dẫn thư mục cục bộ photo for database prj. Nếu thư mục này bị di chuyển sang vị trí khác hoặc bị xóa, giao diện sẽ không hiển thị được hình ảnh sản phẩm. Hệ thống cần được nâng cấp để lưu trữ hình ảnh trên các máy chủ đám mây (Cloud Storage/CDN) và gọi qua đường dẫn URL internet.

2. Khó khăn gặp phải & Giải pháp

STT	Khó khăn gặp phải	Nguyên nhân	Giải pháp khắc phục
1	Bộ đếm tự tăng (Sequence) bị lệch pha sau khi nạp dữ liệu mẫu (báo lỗi <i>duplicate key</i>).	Do nạp dữ liệu mẫu bằng tay trực tiếp kèm các ID khóa chính cố định (Insert explicit IDs), khiến bộ đếm tự tăng (sequence) của PostgreSQL không tự cập nhật theo, dẫn đến sinh trùng ID khi chạy ứng dụng.	Thực thi tệp kịch bản 04_sync_sequences.sql (folder Hướng dẫn cài) sử dụng hàm <code>setval()</code> kết hợp <code>pg_get_serial_sequence()</code> để tự động đặt lại bộ đếm của các bảng về đúng giá trị ID lớn nhất hiện tại.
2	Hardcode cấu hình Database gây lỗi kết nối khi triển khai mã nguồn trên máy khác.	Tên Database, tài khoản kết nối bị viết cố định rải rác ở nhiều lớp giao diện khác nhau, gây ra lỗi FATAL : database does not exist khi chạy trên môi trường PostgreSQL của thành viên khác.	Tái cấu trúc mã nguồn, tập trung toàn bộ cấu hình kết nối vật lý vào một lớp điều khiển duy nhất là DatabaseManager.java . Các lớp giao diện chỉ gọi kết nối gián tiếp qua phương thức tập trung này.
3	Quản lý tính toàn vẹn của giao dịch (Transaction) phức tạp và dễ mất nhất quán dữ liệu khi đặt hàng.	Quy trình checkout gồm nhiều bước liên tiếp: <i>Tạo đơn → Thêm chi tiết (trừ kho) → Gán đối tác → Điều phối shipper → Tạo vận đơn → Xóa giỏ</i> . Nếu lỗi ở giữa luồng sẽ gây mâu thuẫn dữ liệu nghiêm trọng.	Bọc toàn bộ quy trình checkout trong một transaction duy nhất bằng cách thiết lập <code>conn.setAutoCommit(false)</code> . Chỉ thực thi <code>conn.commit()</code> khi tất cả các bước thành công, và gọi <code>conn.rollback()</code> nếu xảy ra lỗi.
4	Khó khăn trong việc lưu trữ và hiển thị tài nguyên hình ảnh của danh mục và sản phẩm.	Lưu trữ trực tiếp dữ liệu hình ảnh nhị phân (Bytea/BLOB) vào PostgreSQL sẽ làm phình to dung lượng ổ đĩa CSDL cực nhanh và làm suy giảm nghiêm trọng tốc độ truy vấn trang chủ.	Áp dụng giải pháp chuẩn thực tế: Lưu trữ toàn bộ tệp hình ảnh tại thư mục photo for database prj nằm ngay trong cấu trúc mã nguồn của ứng dụng. Trong CSDL chỉ lưu trữ đường dẫn tương đối dạng văn bản (VARCHAR). Java Swing sẽ đọc đường dẫn này từ DB để tải ảnh cục bộ lên giao diện nhanh chóng.

3. Phân công nhiệm vụ chi tiết

STT	Thành viên	Nhiệm vụ chi tiết	Kết quả đánh giá
1	Nguyễn Thị Thanh Trúc 202417052 (Nhóm trưởng)	- Phân chia công việc, điều phối nhiệm vụ và kiểm soát tiến độ hoàn thành dự án. - Chịu trách nhiệm viết, tổng hợp, chỉnh sửa và hoàn thiện báo cáo Bài tập lớn. - Khảo sát bài toán nghiệp vụ thực tế, phân tích hệ thống và thiết kế sơ đồ thực thể liên kết ERD. - Thiết lập cấu trúc bảng cơ sở dữ liệu (setup.sql), các ràng buộc toàn vẹn (PK/FK/CHECK) và kiểu dữ liệu ENUM trên PostgreSQL. - Chuyển đổi sơ đồ ERD sang mô hình quan hệ đạt chuẩn 3NF và vẽ sơ đồ quan hệ chi tiết. - Xây dựng phân hệ Java "Đăng ký & Đăng nhập định danh. - Thiết kế cấu trúc hệ thống, tích hợp và đồng bộ hóa các phân hệ giao diện của cả nhóm thành một chương trình hoàn chỉnh (Main.java). - Phát triển công cụ sinh dữ liệu tự động (tools_datagen) sử dụng thư viện Datafaker, gửi kịch bản mẫu cho cả nhóm đề nạp (insert) dữ liệu vào database và viết tập lệnh đồng bộ bộ đếm tự tăng (Sequence). - Viết 10 câu truy vấn + index, view, trigger, function. - Làm slide thuyết trình.	10/10

		- Nhận xét, đánh giá chéo và góp ý hoàn thiện.	
2	Nguyễn Minh Đăng 202416877	- Xây dựng phân hệ Java "Đặt hàng & Điều phối Logistics tự động". - Viết 10 câu truy vấn. - Nhận xét, đánh giá chéo và góp ý hoàn thiện.	5/10
3	Nguyễn Bá Long 202416970	- Xây dựng phân hệ Java "Tìm kiếm & Lọc danh mục sản phẩm phẳng" kết hợp hiển thị lưới thể sản phẩm trực quan. - Xây dựng phân hệ Java "Thêm sản phẩm vào giỏ" (kiểm tra tồn kho bằng Spinner số lượng), chỉnh sửa số lượng trong giỏ và thiết lập Dropdown địa chỉ nhận bằng động. - Tổ chức, liên kết thư mục hình ảnh cục bộ (photo for database prj) và bổ sung tính năng lựa chọn phương thức thanh toán cho ứng dụng. - Viết 10 câu truy vấn. - Nhận xét, đánh giá chéo và góp ý hoàn thiện.	7/10

-----KẾT THÚC-----